

석 사 학 위 논 문

SST 파일 수명을 고려한 ZNS 기반 키-밸류  
스토어의 Zone 관리 최적화

배재홍

부산대학교 대학원  
정보융합공학과

2026년 8월

SST 관측 수평에 따른 ZNS 기반 키-밸류 스토어의 Zone 관리 최적화

배재홍

2026.8

# SST 파일 수명을 고려한 ZNS 기반 키-밸류 스토어의 Zone 관리 최적화

이 논문을 공학석사 학위논문으로 제출함

배재홍

부산대학교 대학원

정보융합공학과

지도교수 안성용

배재홍의 공학석사 학위논문을 인준함

2026년 06월 29일

|     |     |   |
|-----|-----|---|
| 위원장 | 권동현 | 인 |
|-----|-----|---|

|    |     |   |
|----|-----|---|
| 위원 | 손준영 | 인 |
|----|-----|---|

|    |     |   |
|----|-----|---|
| 위원 | 안성용 | 인 |
|----|-----|---|

## 차 례

|                                                |    |
|------------------------------------------------|----|
| I. 서론 .....                                    | 1  |
| 1. 연구의 배경과 목적 .....                            | 1  |
| 가. 연구의 배경 .....                                | 1  |
| 나. 연구의 필요성 .....                               | 1  |
| 다. 연구 목적 및 내용 .....                            | 2  |
| II. 배경 .....                                   | 3  |
| 1. 낸드 플래시 기반 저장 장치 및 ZNS SSD .....             | 3  |
| 가. 낸드 플래시 기반 저장 장치 .....                       | 3  |
| 나. ZNS SSD .....                               | 4  |
| 2. LSM-Tree 및 RocksDB .....                    | 5  |
| 가. LSM-Tree .....                              | 5  |
| 나. RocksDB .....                               | 6  |
| 3. ZenFS .....                                 | 7  |
| III. 관련 연구 .....                               | 10 |
| 1. 데이터 배치 관련 연구 .....                          | 10 |
| 가. D-ZAP .....                                 | 10 |
| 2. Zone reset 관련 연구 .....                      | 10 |
| 가. ReZNS .....                                 | 10 |
| 나. FAR .....                                   | 11 |
| IV. 연구 동기 .....                                | 13 |
| 1. ZenFS의 zone 관리 정책 분석 .....                  | 13 |
| 가. 실험 환경 .....                                 | 13 |
| 나. ZenFS의 zone 할당 정책 분석 .....                  | 14 |
| 다. ZenFS의 zone reset 정책 분석 .....               | 15 |
| V. SLA(Strong Lifetime based Allocation) ..... | 17 |
| 1. SLA 정책의 개념 및 기대 효과 .....                    | 17 |
| 2. SLA 알고리즘 구현 .....                           | 18 |
| VI. LZR(Lazy Zone Reset) .....                 | 20 |
| 1. Lifetime별 SST 파일의 특징 분석 .....               | 20 |

|                                                  |    |
|--------------------------------------------------|----|
| 2. LZR 정책의 개념 및 동작 원리 .....                      | 20 |
| 3. LZR 알고리즘 구현 .....                             | 21 |
| VII. 실험 및 평가 .....                               | 23 |
| 1. 실험 환경 .....                                   | 23 |
| 2. <i>db_bench</i> fillrandom 실험 .....           | 23 |
| 가. 성능 관련 지표 분석 .....                             | 23 |
| 나. 공간 활용 효율성 평가 .....                            | 25 |
| 다. 저장 장치 신뢰성 및 zone reset 효율성 분석 .....           | 26 |
| 3. <i>db_bench</i> mixgraph 실험 .....             | 27 |
| 가. 실험 워크로드 개요 .....                              | 27 |
| 나. 실험 환경 설정 .....                                | 27 |
| 다. 실험 결과 .....                                   | 28 |
| 4. YCSB(Yahoo! Cloud Serving Benchmark) 실험 ..... | 29 |
| 가. YCSB 실험 구성 및 워크로드 특성 .....                    | 29 |
| 나. 실행 시간 분석 및 가비지 컬렉션의 영향 .....                  | 30 |
| VIII. 결론 .....                                   | 32 |
| 참고문헌 .....                                       | 33 |
| Abstract .....                                   | 37 |

## 표 차 례

|                                          |    |
|------------------------------------------|----|
| [표 1] 실험 환경 .....                        | 13 |
| [표 2] <i>db_bench</i> 옵션 .....           | 13 |
| [표 3] SLA를 적용했을 때 reset된 zone의 사용률 ..... | 20 |
| [표 4] YCSB 워크로드 특성 .....                 | 29 |

## 그 립 차 례

|                                                               |    |
|---------------------------------------------------------------|----|
| [그림 1] SSD와 ZNS SSD 비교 .....                                  | 4  |
| [그림 2] RocksDB 및 LSM-Tree 구조 .....                            | 5  |
| [그림 3] 시스템 아키텍처 개요 .....                                      | 7  |
| [그림 4] ZenFS의 Zone 할당 정책 수도 코드 .....                          | 8  |
| [그림 5] ZenFS의 Zone reset 정책 수도 코드 .....                       | 9  |
| [그림 6] ReZNS 개요 .....                                         | 11 |
| [그림 7] FAR의 zone reset 방식 .....                               | 12 |
| [그림 8] db_bench 수행 중 사용 중인 zone의 비율과 ZNS SSD의 여유 공간 비율 ·      | 14 |
| [그림 9] 워크로드 진행 정도에 따른 ZNS SSD의 내부 상태 .....                    | 14 |
| [그림 10] 워크로드 진행 정도에 따른 zone reset의 발생 .....                   | 15 |
| [그림 11] 각 정책별 ZNS SSD의 내부 상태 및 쓰기 요청 처리 결과 .....              | 17 |
| [그림 12] 각 정책별 reset 가능 zone .....                             | 18 |
| [그림 13] SLA 수도 코드 .....                                       | 18 |
| [그림 14] SLA와 함께 적용한 zone reset 정책 .....                       | 21 |
| [그림 15] LZR 수도 코드 .....                                       | 21 |
| [그림 16] (a) 데이터 복사량 비교 (b) 처리량 비교 (c) 쓰기 중단 발생 시간 비교 ····     | 23 |
| [그림 17] (a) 실행 시간에 따른 사용 zone 비율 (b) 벤치마크 종료 후 사용 zone 수      | 25 |
| [그림 18] zone reset 횟수 비교 .....                                | 26 |
| [그림 19] mixgraph의 (a) 각 키에 접근될 확률 (b) 밸류 크기 분포 .....          | 27 |
| [그림 20] (a) 쓰기 중단 시간 비교 (b) zone reset 횟수 비교 (c) 처리량 비교 ····· | 28 |
| [그림 21] YCSB 실행 시간 .....                                      | 30 |

# SST 파일 수명을 고려한 ZNS 기반 키-밸류 스토어의 Zone 관리 최적화

배재홍

부산대학교 대학원, 정보융합공학과

## 요약

현대 데이터 집약적 애플리케이션의 확산으로 인해 고성능 저장 장치에 대한 수요가 증가함에 따라, 기존 블록 인터페이스의 한계를 극복하기 위한 차세대 스토리지 인터페이스인 ZNS(Zoned Namespace) SSD가 주목받고 있다. 특히 Facebook(Meta)에서 개발한 RocksDB는 LSM-Tree 구조를 바탕으로 높은 쓰기 성능을 제공하며 SSD를 위한 대표적인 스토리지 엔진으로 널리 활용되고 있다. 그러나 RocksDB의 ZNS 전용 스토리지 백엔드인 ZenFS는 경직된 zone 관리 정책으로 인해 물리적 공간이 충분함에도 사용 가능한 zone이 조기에 고갈되는 zone 할당 실패가 발생하며, 이는 시스템의 가용성과 장치 신뢰성을 저해하는 요소로 지적된다.

본 논문에서는 이러한 한계를 해결하기 위해 데이터의 수명 특성을 고려한 사용자 레벨의 새로운 zone 관리 정책인 SLA-LZR을 제안한다. 제안된 기법은 크게 두 가지 핵심 정책으로 구성된다. 첫째, Strong Lifetime Allocation(SLA) 정책은 동일한 수명을 가진 SST(Sorted String Table) 파일들을 단일 zone 내에 집적하여 배치함으로써 데이터 무효화 시점을 일치시키고, zone reset 효율을 극대화하여 사용 가능한 zone을 적시에 확보한다. 둘째, Lazy Zone Reset(LZR) 정책은 RocksDB 로그 파일의 단기 수명 특성을 반영하여, 유효 데이터가 없더라도 zone이 완전히 채워지지 않은 경우 reset을 지연시키는 적응형 메커니즘을 통해 ZNS SSD의 불필요한 장치 마모를 방지한다.

제안한 정책의 실효성을 검증하기 위해 FEMU 기반의 ZNS SSD 에뮬레이션 환경을 구축하고 RocksDB의 *db\_bench*와 YCSB 벤치마크를 수행하였다. 실험 결과, SLA-LZR은 가비지 컬렉션(Garbage Collection) 과정에서 발생하는 데이터 복사량을 기존 ZenFS 대비 약 93.5% 감소시켰으며, 이를 통해 전체 시스템 처리량을 40.6% 향상시켰다. 또



한 스토리지 I/O 블로킹을 최소화하여 쓰기 지연(Write Stall) 현상을 13.8% 완화하였으며, 최종적으로 zone reset 횟수를 43.7% 감소시킴으로써 장치의 신뢰성 및 기대 수명을 유의미하게 연장하였다. 특히 실제 워크로드 기반 벤치마크인 YCSB 평가에서는 모든 실험 과정에서 단 한 차례의 가비지 컬렉션도 유발하지 않는 최적의 자원 관리 성능을 입증하였다.

결론적으로 본 연구는 애플리케이션 계층에서 데이터의 수명 정보를 스토리지 관리 정책에 효과적으로 반영함으로써, ZNS SSD 기반 차세대 스토리지 시스템의 성능과 수명을 동시에 개선할 수 있는 실질적인 방안을 제시하였다.

# I. 서론

## 1. 연구의 배경과 목적

### 가. 연구의 배경

최근 데이터 센터와 클라우드 컴퓨팅 환경에서 발생하는 데이터의 폭발적인 증가는 대규모 데이터를 효율적으로 관리하기 위한 키-밸류 스토어(Key-Value Store)의 중요성을 증대시켰다[1, 2, 3]. 특히 Facebook(Meta)에서 개발한 RocksDB는 LSM-Tree(Log Structured Merge-Tree) 구조를 바탕으로 높은 쓰기 성능을 제공하며 현대 스토리지 엔진의 표준으로 자리 잡았다[4, 5, 6, 7]. 특히 순차 쓰기(Sequential Write)를 통해 플래시 메모리의 out-of-place 업데이트 특성과 잘 맞아떨어져 플래시 기반 장치인 SSD에서 높은 I/O 성능을 나타낸다[8, 9]. 그러나 잦은 컴팩션(Compaction)으로 인해 데이터 무효화(Invalidation) 및 쓰기가 빈번하게 발생하여 SSD의 수명과 성능에 영향을 미치고 가비지 컬렉션(Garbage Collection)을 유발한다[10, 11].

그러나 기존의 블록 인터페이스 기반의 SSD(Solid State Drive)는 장치 내부의 FTL(Flash Translation Layer)로 인해 예측 불가능한 지연 시간과 가비지 컬렉션에 따른 쓰기 증폭(Write Amplification) 문제를 가지고 있다. 이러한 한계를 극복하기 위해 등장한 ZNS(Zoned Namespace) SSD는 호스트가 직접 데이터를 배치하고 가비지 컬렉션을 수행한다. 그리고 낸드 플래시 메모리의 물리적 특성에 맞춰 순차 쓰기(Sequential Write)를 강제함으로써 효율성을 극대화한다.

### 나. 연구의 필요성

ZNS SSD를 RocksDB와 함께 사용하기 위해 제안된 사용자 레벨 파일 시스템 플러그인(User-Level File System Plugin) ZenFS는 LSM-Tree의 각 레벨 구조와 컴팩션 특성을 반영한 zone 관리 정책을 통해 RocksDB의 쓰기 성능을 최적화한다. 그러나 현재 ZenFS가 채택하는 zone 관리 정책은 실제 환경에서 다음과 같은 한계점을 가진다.

첫째, 수명 기반 Zone 할당 정책(Lifetime based Zone Allocation Policy)으로 인해 자원 관리의 비효율성이 발생한다. ZenFS는 가장 높은 수명의 SST(Sorted-String Table) 파일을 쓰기 위해 항상 새로운 빈(Empty) zone을 할당한다. 이로 인해 ZNS SSD 내부의 물리적인 전체 여유 공간은 충분함에도 불구하고 사용 가능한 빈 zone의

개수가 빠르게 고갈되는 현상이 발생한다. 이는 가용 자원의 불균형을 초래하여 시스템의 지속적인 쓰기 작업을 방해하는 요소가 된다.

둘째, 무분별한 Zone Reset 정책으로 인해 ZNS SSD의 신뢰성이 저하된다. 현 ZenFS는 사용 가능한 zone 및 공간을 확보하기 위해 무효(Invalid) 데이터만 존재하는 zone을 찾아 즉각적으로 reset하는 정책을 사용하고 있다. 그러나 이 과정에서 해당 zone의 사용률(Utilization)을 충분히 고려하지 않아 극히 일부만 사용된 zone까지 빈번하게 reset될 수 있다. 이는 ZNS SSD의 불필요한 P/E Cycle(Program/Erase Cycle)을 증가시켜 장치의 물리적 수명을 단축시키고 장치의 신뢰성을 떨어뜨릴 수 있다.

셋째, 데이터 배치(Data Placement)로 인한 Zone Reset 정책의 효율성이 저하되는 문제가 발생한다. ZenFS는 비슷한 수명의 SST 파일을 같은 zone에 배치하여 가비지 컬렉션을 수행 시 데이터 복사(Migration) 오버헤드를 낮추려 하지만 수명이 매우 긴 데이터가 특정 zone을 장기간 점유함으로써 reset이 불가능한 상태로 남게 된다. 결과적으로 이는 효율적인 zone reset 메커니즘의 작동을 방해하며 사용 가능한 zone 부족에 따른 가비지 컬렉션을 유발하여 시스템 전반의 I/O 지연 시간(Latency)과 처리량(Throughput)에 악영향을 미친다.

#### 다. 연구 목적 및 내용

본 논문에서는 RocksDB 및 ZenFS 환경에서 ZNS SSD의 공간 활용 효율과 장치 수명을 최적화하기 위한 새로운 zone 관리 정책을 제안한다. 본 연구의 구체적인 제안 사항으로, 첫 번째로 SLA(Strong Lifetime based Allocation)를 제안한다. SLA는 수명별로 데이터를 분류해 같은 수명의 데이터를 같은 zone에 배치하는 정책이다. 이를 통해 동일 zone 내 데이터 무효화 시점을 일치시켜 zone reset의 효율을 극대화하고 가비지 컬렉션 비용을 최소화한다. 두 번째로 데이터 수명과 zone의 사용률을 모두 고려한 LZR(Lazy Zone Reset)을 제안한다. LZR은 사전에 확인한 각 zone이 reset될 때의 사용률과 수명별 SST 파일의 특징을 바탕으로 각 수명별 zone reset 방식을 달리한다.

결론적으로 본 연구는 ZenFS의 한계였던 zone 고갈과 수명 저하를 해결하고 가비지 컬렉션으로 인한 복사 오버헤드를 줄임으로써 차세대 스토리지인 ZNS SSD 기반 키-밸류 스토어의 실용성을 높이는 데 기여하고자 한다.

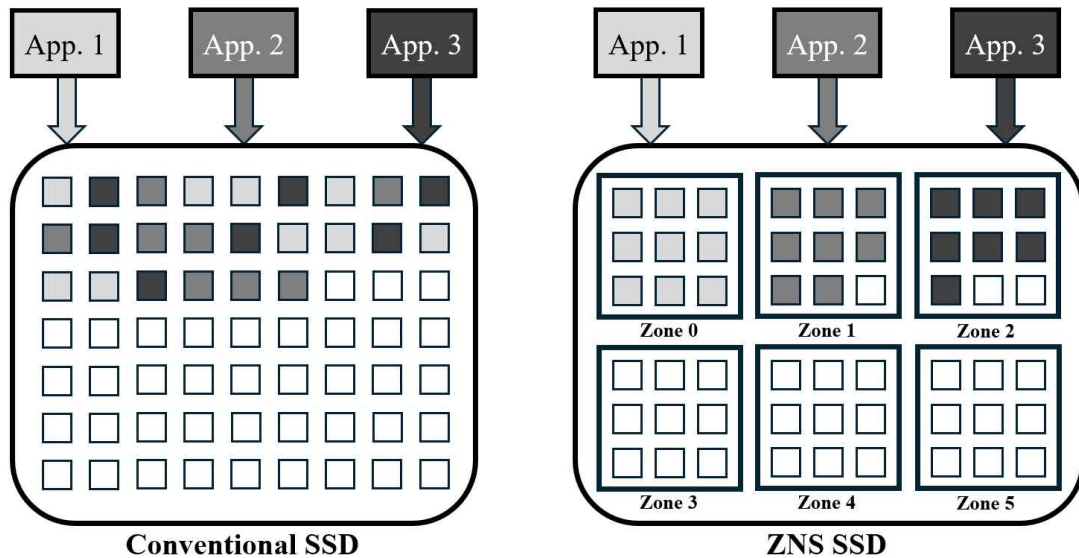
## II. 배경

### 1. 낸드 플래시 기반 저장 장치 및 ZNS SSD

#### 가. 낸드 플래시 기반 저장 장치

NAND 플래시 메모리는 비휘발성(Non-Volatile) 저장 장치로, 데이터를 저장하는 최소 단위인 셀(Cell)들로 구성된다. 이러한 물리적 구조로 인해 일반적인 자기 디스크(HDD)와는 다른 제약 사항을 가진다. 첫 번째는 Erase-Before-Write(쓰기 전 삭제)로, 플래시 메모리는 이미 데이터가 기록된 셀에 바로 덮어쓰기(Overwrite)를 할 수 없다. 새로운 데이터를 기록하기 위해서는 반드시 해당 영역을 먼저 삭제(Erase)해야 한다. 두 번째는 읽기/쓰기와 삭제 단위가 일치하지 않는 것이다. 데이터의 읽기와 쓰기는 페이지(Page) 단위로 이루어지는 반면, 삭제는 여러 페이지가 묶인 블록(Block) 단위로 수행된다. 이 불일치는 유효한 데이터가 남은 블록을 삭제하기 위해 데이터를 다른 곳으로 복사해야 하는 비효율을 초래한다. 마지막 제약 사항은 제한된 수명을 가지고 있다는 것이다. 플래시 메모리 셀은 데이터를 쓰고 지우는 과정에서 전기적인 신호를 사용하기 때문에 데이터를 지울 때마다 산화막이 마모되어 셀의 물리적인 수명이 줄어든다. 이로 인해 플래시 메모리는 제한적인 P/E Cycle(Program/Erase Cycle) 횟수를 가지며, 제한된 횟수를 초과하면 해당 셀은 더 이상 사용할 수 없게 된다. 위와 같은 제약 사항으로 인해 SSD는 호스트 시스템이 플래시 메모리를 일반적인 하드디스크처럼 인식할 수 있도록 FTL(Flash Translation Layer)이라는 중간 관리 계층을 둔다. FTL은 논리 주소를 물리 주소로 매핑하고, 셀의 마모도를 균등하게 분산시키는 웨어 레벨링(Wear Leveling) 및 SSD 내의 무효 데이터를 일괄적으로 삭제하는 가비지 컬렉션을 수행한다. 그러나 FTL의 내부 관리 방식은 다음과 같은 비용을 초래한다. 첫 번째 비용은 쓰기 증폭이다. 가비지 컬렉션 과정에서 유효한 데이터를 유지하기 위해 내부적인 복사가 발생한다. 이는 호스트가 요청한 쓰기량보다 실제 플래시 메모리에 기록되는 양을 증가시켜 장치의 성능 저하와 수명 단축을 초래한다. 두 번째 비용은 성능 예측의 불가능성이다. FTL이 백그라운드에서 임의로 가비지 컬렉션을 수행할 때 호스트의 I/O 요청이 들어오거나 I/O 수행 중 가비지 컬렉션이 필요할 경우 급격한 지연 시간 상승(Tail Latency)을 유발한다[22, 23, 24].

## 나. ZNS SSD



[그림 1] SSD와 ZNS SSD 비교

이러한 블록 인터페이스 SSD의 한계를 극복하고, 호스트가 직접 플래시 메모리의 물리적 구역을 관리하여 효율성을 높이기 위해 ZNS SSD가 제안되었다.

[그림 1]은 기존 SSD와 ZNS SSD를 비교한 그림이다. 기존 블록 인터페이스 SSD는 호스트가 데이터를 임의의 위치에 기록할 수 있는 논리 블록 주소(LBA, Logical Block Address) 방식을 사용한다. 이 방식은 사용이 편리하지만, SSD 내부의 FTL이 물리적 배치를 전적으로 결정하기 때문에 데이터의 실제 위치를 호스트가 알 수 없다는 블랙박스 구조를 가진다.

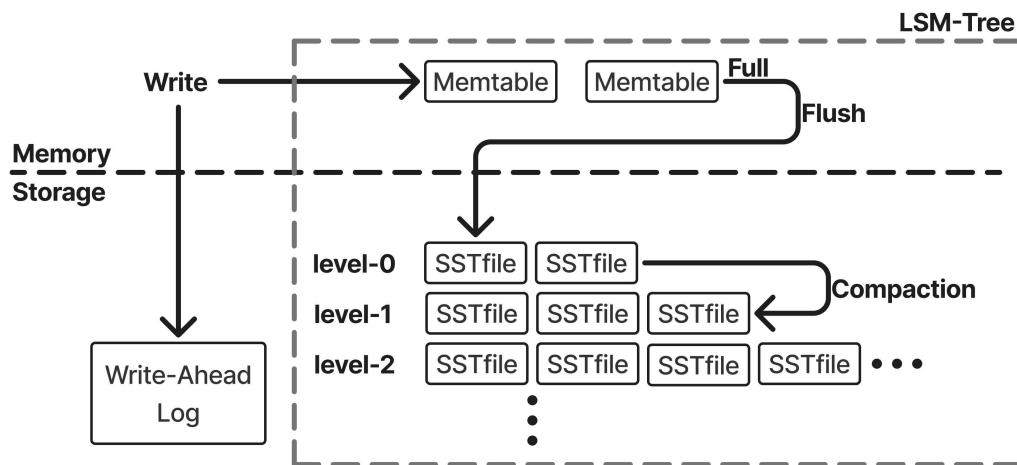
반면, ZNS SSD는 전체 저장 공간을 일정한 크기의 zone이라는 논리 단위로 나눈다. 각 zone 내에서는 반드시 순차 쓰기(Sequential Write)만 허용되며, 데이터를 수정하려면 해당 zone 전체를 reset해야 한다. 이러한 제약은 기존 SSD에 비해 더 많은 제약을 가진 것처럼 보이지만, 호스트(Host)가 특정 명령을 통해 데이터를 쓰고 zone을 reset할 수 있어 기존 FTL이 수행하던 데이터 배치 및 가비지 컬렉션 작업을 호스트로 일임할 수 있다[13, 14, 25]. 그리고 호스트는 FTL 수준에서는 알 수 없는 데이터의 수명과 같은 정보를 알 수 있어 해당 정보를 바탕으로 데이터를 각 zone에 분리 배치해 가비지 컬렉션 오버헤드를 최소화할 수 있다[12, 26].

그리고 ZNS SSD는 기존의 블록 인터페이스가 아닌 zone 인터페이스를 사용한다.

이로 인해 기존의 블록 인터페이스 파일 시스템을 사용할 수 없고 전용의 파일 시스템이나 zone을 인식할 수 있는 소프트웨어가 필요하다.

## 2. LSM-Tree 및 RocksDB

### 가. LSM-Tree



[그림 2] RocksDB 및 LSM-Tree 구조

[그림 2]의 점선 내부는 LSM-Tree의 대략적인 구조를 나타낸다. LSM-Tree는 데이터를 직접 수정하는 대신 새로운 데이터를 계속해서 쌓아 나가는 추가 전용(Append only) 구조를 가지는 자료 구조이다. 이는 임의 쓰기(Random Write)를 순차 쓰기(Sequential Write)로 처리하여 스토리지의 쓰기 처리량을 극대화하는 방식이다[8, 9].

데이터를 쓰는 과정은 크게 두 단계로 나뉜다. 첫 번째는 MemTable(In-memory)에 쓰기로 모든 쓰기 요청은 먼저 메모리 상의 자료구조인 MemTable에 써진다. 이 때 키-밸류 쌍은 키에 따라 정렬되지 않고 들어오는 순서에 따라 그대로 저장된다. 두 번째는 SST 파일(In-storage)로 MemTable이 일정 크기에 도달하면 불변(Immutable) 상태로 플러시(Flush)되어 실제 저장 장치에 SST 파일 형태로 저장된다[4, 5].

SST 파일들은 계층적으로 구성된 여러 논리적인 레벨(L0, L1, ..., Ln)로 관리된다. L0(Level 0)에는 MemTable이 플러시되어 생성된 SST 파일들이 저장된다. L0의 SST

파일들은 MemTable의 데이터를 그대로 내려쓴 것이므로 하나의 정렬된 런(Sorted Run)을 이루지 못하며, 파일 간에 키 범위가 서로 겹칠 수 있다. 반면 L1 이하의 레벨은 전체 키 범위가 정렬된 하나의 런으로 구성되어 여러 SST 파일로 분할되며, 동일 레벨 내에서는 파일 간 키 범위가 겹치지 않는다. 또한 레벨이 커질수록(하위 레벨로 갈수록) 수용 가능한 데이터 용량이 일정 배수(default 10배)씩 지수적으로 증가한다.

LSM-Tree는 읽기 성능 유지와 공간 회수를 위해 컴팩션(Compaction)이라는 작업을 수행한다. 컴팩션은 상위 레벨의 파일을 하위 레벨의 파일과 병합하여 중복되거나 삭제된 데이터를 제거하는 과정이다. 본 연구에서 가장 중요한 점은 레벨에 따른 데이터의 수명(Lifetime) 차이이다. 상위 레벨 (L0, L1)은 용량 임계치가 작아 컴팩션이 매우 빈번하게 발생한다. 데이터가 빠르게 하위 레벨로 이동하거나 삭제되므로 짧은 수명을 가진다. 하위 레벨 (~ Ln)로 갈수록 용량 임계치가 커지며, 데이터가 한 번 배치되면 상위 레벨에서 컴팩션 조건이 충족될 때까지 오랜 기간 머무르게 된다. 이러한 데이터는 긴 수명을 가진다.

LSM-Tree의 SST 파일은 생성 시점에 모든 데이터가 정렬되어 순차적으로 써지고 한 번 생성되면 컴팩션이 발생하기 전까지 수정되지 않으며 삭제되지 않는 불변성을 가진다. 그리고 각 레벨에 따른 데이터 수명 차이도 분명하다. 이러한 특성은 순차 쓰기만을 허용하고 덮어쓰기가 불가능한 ZNS SSD의 제약과 완벽히 부합하고 호스트 레벨에서 데이터 수명을 예측하여 zone을 할당하기에 유리한 구조를 가진다.

## 나. RocksDB

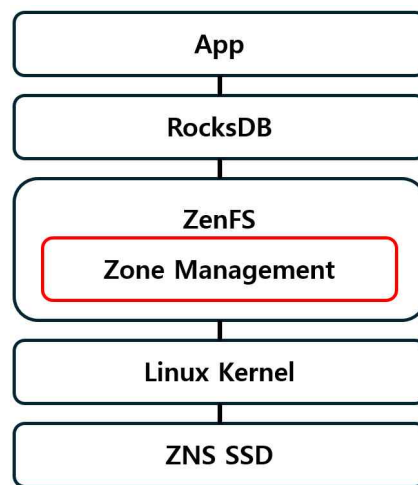
RocksDB는 Facebook(Meta)에서 개발한 오픈 소스 키-밸류 스토어(Key-Value Store)로, Google의 LevelDB를 기반으로 빠른 스토리지(SSD, ZNS SSD)의 성능을 최대한 활용하도록 설계되었다. RocksDB는 데이터 저장 엔진으로서 높은 확장성과 유연한 설정 기능을 제공하며, 현재 분산 데이터베이스, 메타데이터 관리 서비스 등 고성능 I/O 처리가 요구되는 다양한 시스템의 하부 스토리지 엔진으로 널리 사용되고 있다[33].

[그림 2]는 LSM-Tree 기반의 RocksDB의 구조를 나타낸다. RocksDB는 데이터 관리를 위해 다음과 같은 구성 요소를 가진다. MemTable과 SST 파일은 앞서 LSM-Tree에서 설명한 부분과 같다[4, 5]. 이외 추가적인 요소인 Write-Ahead Log(WAL)는 시스템 장애 시 휘발성 메모리인 주 메모리(Main Memory)에 쓰인 MemTable의 데이터가 소실될 것을 대비해 모든 쓰기 요청을 비휘발성 메모리인 저장 장치(SSD, ZNS SSD)에

순차적으로 기록하는 로그 파일이다. WAL은 MemTable이 SST 파일로 플러시 되면 안전하게 삭제된다.

그리고 RocksDB는 레벨별 데이터의 수명을 제공한다. WAL은 모든 파일 중 가장 빠르게 삭제되는 특징을 가지므로 가장 짧은 수명인 2를 부여받는다. 그리고 컴팩션이 빈번하게 발생하는 L0, L1의 SST 파일은 다음으로 짧은 수명인 3을 부여받는다. 다음으로 L2 SST 파일의 수명은 4, L3 이상의 SST 파일의 수명은 5이다.

### 3. ZenFS



[그림 3] 시스템 아키텍처 개요

[그림 3]은 ZenFS가 적용된 전체 시스템 개요를 보여준다. ZenFS는 RocksDB가 ZNS SSD의 이점을 활용할 수 있도록 설계된 사용자 레벨 파일 시스템이다[18, 19]. 이는 기존의 블록 레이어 추상화로 인해 발생하는 성능 저하와 쓰기 증폭을 해결하기 위해, RocksDB의 파일 시스템 인터페이스를 직접 구현하여 호스트 수준에서 ZNS SSD의 zone을 직접 관리할 수 있도록 설계되었다.

본 논문에서 중점적으로 다루는 zone 관리 정책은 데이터를 쓸 때 최적의 zone을 할당하는 'zone 할당 정책'과 사용 가능한 zone을 선제적으로 확보하여 가비지 컬렉션 발생을 지연시키는 'zone reset 정책'이다. ZenFS는 RocksDB에서 생성되는 SST 파일들을 ZNS SSD의 순차 쓰기 전용 zone에 수명별로 배치함으로써, 장치 내부의 백그라운드(Background) 가비지 컬렉션을 제거하고 시스템 처리량 및 ZNS SSD의 수명을 확보하는 것을 목적으로 한다.



---

**Algorithm 1** ZoneAllocate

---

**Require:** SST\_lifetime

```
1: best_diff = 100, alloc_zone = null
2: for all z ∈ io_zones do
3:   life_diff = z.lifetime - SST_lifetime
4:   if z.capacity > 0 and z.lifetime > SST_lifetime and best_diff ≥ life_diff
     then
5:     alloc_zone = z
6:     best_diff = life_diff
7:   end if
8: end for
9: if alloc_zone == null then
10:  alloc_zone = AllocateNewZone()
11:  alloc_zone.lifetime = SST_lifetime
12: end if
13: return alloc_zone
```

---

[그림 4] ZenFS의 Zone 할당 정책 수도 코드

[그림 4]는 ZenFS의 zone 할당 정책을 수도 코드(Pseudo Code)로 나타낸 것이다. 현재 ZenFS는 RocksDB로부터 전달받은 수명 힌트(Lifetime Hint)를 기반으로 zone을 할당하여 데이터를 배치한다. 먼저 zone의 수명 힌트는 해당 zone에 처음으로 쓰인 SST 파일의 수명이 된다. SST 파일의 쓰기 요청이 발생하면 ZenFS는 해당 SST 파일의 수명 힌트와 전체 zone들의 수명 힌트를 비교한다. 그리고 SST 파일의 수명보다 길면서도 그 차이가 가장 적은 zone을 할당한다. 즉, 같은 수명의 zone은 할당 대상에서 제외한다. 그리고 수명이 더 긴 zone이 없거나 수명 조건을 만족하는 zone들 모두 여유 공간이 부족한 경우 새로운 zone을 할당하고 해당 zone의 수명은 SST 파일의 수명으로 설정된다.

이러한 할당 전략의 핵심은 수명이 유사한 SST 파일들을 동일한 zone에 모으는 데 있다. 즉, 특정 zone에 최초로 쓰인 SST 파일보다 수명이 짧으면서도 이에 근접한 데이터들을 모아 배치함으로써, zone 내의 모든 데이터가 비슷한 시기에 무효화되도록 유도한다. 이는 별도의 데이터 복사 없이 zone 전체를 reset할 수 있는 확률을 높이거나 가비지 컬렉션 발생 시 zone에 남게 되는 유효 데이터의 양을 최소화하여 복사 비용 및 시스템 오버헤드를 줄이는 것을 목적으로 한다.

---

**Algorithm 2** ZoneReset

---

```
1: for all  $z \in \text{io\_zones}$  do  
2:   if  $z.\text{wp} \neq z.\text{start}$  and  $z.\text{used\_capacity} == 0$  then  
3:     ResetZone( $z$ )  
4:   end if  
5: end for
```

---

[그림 5] ZenFS의 Zone Reset 정책 수도 코드

또한, ZenFS는 시스템의 사용 가능한 공간을 확보하기 위해 zone reset 정책을 사용한다. [그림 5]는 이러한 zone reset 알고리즘의 동작 과정을 수도 코드로 나타낸 것이다. ZenFS는 특정 시점마다 zone reset 알고리즘을 실행하여 유효 데이터가 없는 zone을 reset함으로써 여유 공간 및 사용 가능한 zone을 확보한다. 알고리즘이 실행되는 주요 원인은 컴팩션 등의 작업으로 인해 SST 파일이 삭제되어 zone 내의 데이터가 무효화되는 경우이다.

zone reset 알고리즘이 실행되면 ZenFS는 ZNS SSD 내의 모든 zone을 스캔하여 각 zone을 확인한다. 확인 결과, 특정 zone에 포함된 모든 SST 파일이 삭제되어 유효 데이터가 전혀 남지 않은 상태이면 해당 zone을 reset 한다. 이를 통해 시스템은 빈 zone과 여유 공간을 선제적으로 확보하며 가비지 컬렉션을 최대한 지연시키고자 한다.

추가적으로 ZenFS는 ZNS SSD의 전체 여유 공간이 20% 미만일 때 가비지 컬렉션을 시도하여 여유 공간을 확보한다. 가비지 컬렉션은 유효 데이터가 임계값 이하인 zone을 대상으로 진행되며 임계값은 ZNS SSD의 여유 공간이 적을수록 커진다.

### III. 관련 연구

#### 1. 데이터 배치 관련 연구

##### 가. D-ZAP

2025년 ISPD에 게재된 "D-ZAP: Access Frequency-Aware Dynamic Zone Allocation Optimization for Zoned Storage Devices"는 ZNS SSD의 쓰기 증폭 및 가비지 컬렉션 오버헤드를 줄이기 위한 '데이터 온도 인식 기반 동적 zone 할당' 메커니즘을 제안하였다[31]. 해당 논문은 기존 ZNS SSD의 정적인 zone 할당 방식이 워크로드의 가변성을 반영하지 못해 발생하는 비효율성에 주목한다. 구체적으로, 데이터의 접근 빈도나 최신 정도(Recency)를 고려하지 않고 서로 다른 수명을 가진 데이터를 동일한 zone에 혼합 저장할 경우, 잦은 데이터 무효화와 이로 인한 불필요한 데이터 이동이 발생하여 시스템 전체의 쓰기 성능을 저하시키고 장치의 수명을 단축시킨다고 분석했다.

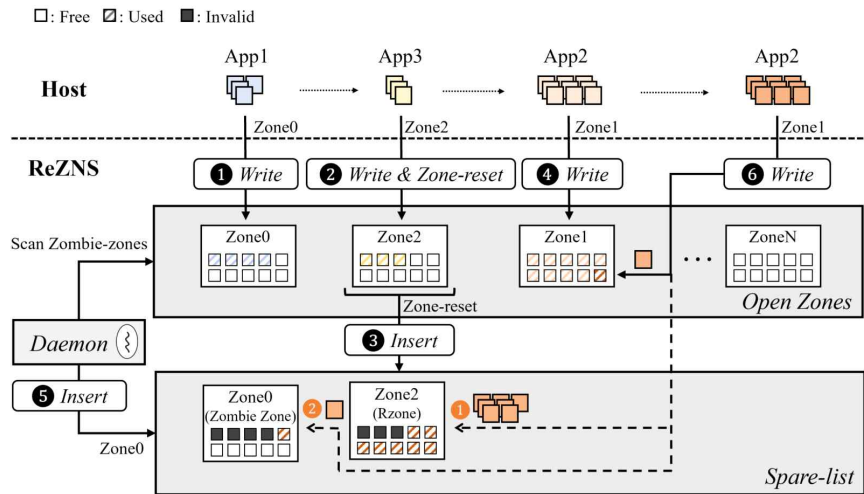
이러한 한계를 극복하기 위해 D-ZAP은 데이터의 특성에 따라 zone을 관리하는 세 가지 전략을 제시한다. 첫째, 데이터 온도 모니터링을 통해 유입되는 데이터의 접근 빈도와 최신 정도를 실시간으로 분석하여 'Hot', 'Warm', 'Cold' 데이터로 분류한다. 둘째, 온도 기반 동적 zone 할당 전략을 통해 분류된 온도 등급에 따라 독립적인 zone 풀(Pool)에 데이터를 배치함으로써 수명이 유사한 데이터끼리 그룹화한다. 셋째, 무효 페이지 비율이 높은 zone을 우선적으로 선택하여 처리하는 적응형 GC 컨트롤러를 도입하여 유효 데이터 복사 비용을 최소화하였다. 실험 결과, D-ZAP은 기존 방식 대비 쓰기 증폭을 최대 51.2%까지 감소시켰으며 처리량과 지연 시간 등 전반적인 성능 지표에서 우수성을 입증하였다.

#### 2. Zone reset 관련 연구

##### 가. ReZNS

2024년에 *Applied Sciences*에 게재된 "ReZNS: Energy and Performance-Optimal Mapping Mechanism for ZNS SSD"는 ZNS SSD의 성능 저하와 에너지 낭비의 핵심 원인을 분석하고, 이를 해결하기 위한 '재생 가능한 zone 매핑' 메커니즘을 제안하였다[32]. 해당 논문은 기존 ZNS SSD 운용 방식에서 발생할 수 있

는 비효율성에 주목한다. 구체적으로 zone 내부에 잔여 공간이 많음에도 불구하고 zone reset 명령이 수행될 경우 불필요한 삭제(Erase) 작업으로 인해 ZNS SSD의 수명을 단축시키고 전력 소비를 가중시킨다는 점을 지적한다. 또한, 응용 프로그램이 할당받은 zone을 모두 채우지 않은 채로 종료되거나 방치된 zone이 ZNS SSD 내부 유효 공간을 낭비하고 시스템 전체 공간 효율성을 저해한다고 분석했다.



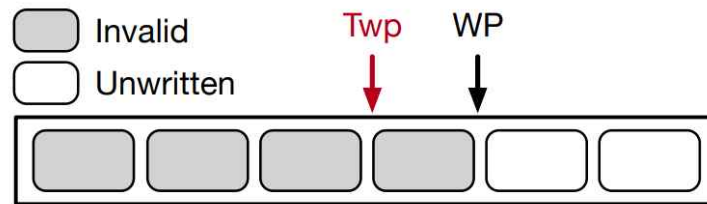
[그림 6] ReZNS 개요

이러한 한계를 극복하기 위해 ReZNS는 잔여 용량을 기반으로 zone을 재사용하는 두 가지 전략을 제시한다. 첫째, zone reset 명령이 내려진 zone의 잔여 용량을 확인하여 25% 이상의 여유 공간이 남았다면 즉시 reset하지 않고 Spare-list에 보관하여 재사용 대기 상태로 전환한다([그림 6]의 2, 3). 둘째, 백그라운드 데몬(Background Daemon)을 통해 응용 프로그램 종료 후 방치된 zone들을 주기적으로 스캔하고 Sparse-list로 회수하는 방식을 도입했다([그림 6]의 5). 새로운 zone 할당 요청이 발생하면 새 zone을 할당하는 대신 Spare-list 내에서 잔여 용량이 가장 큰 zone을 우선적으로 할당한다([그림 6]의 6). 결과적으로 ReZNS는 ZNS SSD의 성능과 수명 그리고 에너지 효율 측면에서 효과를 입증하였다.

#### 나. FAR

2023년 HotStorage에 게재된 “A Free-Space Adaptive Runtime Zone-Reset Algorithm for Enhanced ZNS Efficiency”는 ZNS SSD의 성능을 유지하면서도 장치의 수명을 최적화하기 위한 ‘Free-Space Adaptive Runtime Zone-Reset(FAR)’ 알고

리즘을 제안하였다[20]. 해당 논문은 RocksDB의 ZenFS에서 사용되는 기존의 런타임 zone reset 방식이 장치의 수명 제약을 충분히 고려하지 않는다는 점에 주목한다. 구체적으로, 모든 데이터가 무효화된 zone을 발견 즉시 reset하는 방식은 가비지 컬렉션 호출을 늦춰 성능에는 유리하지만, 불필요하게 잦은 삭제 작업을 유발하여 ZNS SSD의 P/E 사이클을 과도하게 소모시키고 장치 수명을 단축시킨다는 점을 지적한다.



[그림 7] FAR의 zone reset 방식

이러한 한계를 극복하기 위해 FAR은 ZNS SSD 내부의 남은 여유 공간 비율에 따라 zone 리셋 빈도를 동적으로 조절하는 전략을 제시한다. 첫째, 여유 공간 기반 동적 임계값 설정 메커니즘을 도입하여 여유 공간이 충분할 때는 reset 임계값( $T_{wp}$ )을 높게 설정해 삭제 작업을 최대한 지연시키고, 공간이 부족해질수록 임계값을 낮추어 적극적으로 가용 공간을 확보한다. [그림 7]과 같이 zone의 쓰기 포인터(WP)가 임계값을 넘는 경우에만 reset을 수행한다. 둘째, 선형(Linear), 로그(Log), 지수(Exponential) 함수 등 다양한 적응형 함수 모델을 적용하여 성능 저하의 주원인인 쓰기 중단 현상을 방지하면서도 장치의 물리적 마모를 최소화할 수 있는 최적의 reset 타이밍을 결정한다. 결과적으로 FAR은 기존 ZenFS 대비 ZNS SSD의 수명을 약 2배 향상시키면서도 응용 프로그램의 처리량을 안정적으로 유지하는 효과를 입증하였다.

하지만 FAR은 ZNS SSD의 여유 공간이 충분하면  $T_{wp}$ 가 높아 reset을 미루게 되는데, 이는 새로운 zone 확보 불가로 이어진다. 이는 가장 긴 수명의 SST 파일이 써질 때 더 이상 할당할 빈 zone이 부족해지는 결과로 이어진다. 만약 할당할 zone이 없는 경우 FAR은 수명과 무관하게 빈 공간이 있는 아무 zone을 할당하여 SST 파일을 쓴다. 이러한 할당 방식은 여러 수명의 SST 파일이 한 zone에 혼재하여 가비지 컬렉션을 가속화한다.

## IV. 연구 동기

### 1. ZenFS의 zone 관리 정책 분석

#### 가. 실험 환경

[표 1] 실험 환경

|         |                 |                      |
|---------|-----------------|----------------------|
| Host    | CPU             | Intel Xeon Gold 5218 |
|         | Memory          | 128GB                |
| FEMU    | Core            | 32                   |
|         | Memory          | 16GB                 |
|         | Kernel          | Linux 5.10.179       |
| ZNS SSD | Size            | 102.4GB              |
|         | Zone Size       | 512MB                |
|         | Number of Zones | 200                  |

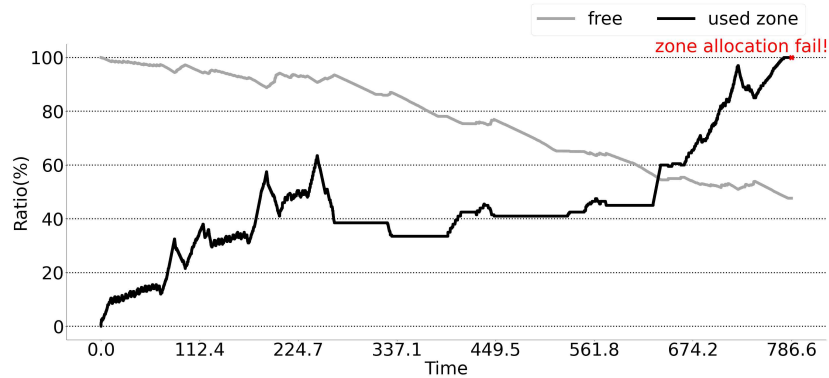
[표 2] *db\_bench* 환경 설정

|                  |             |
|------------------|-------------|
| workload         | fillrandom  |
| key / value size | 16B / 1024B |
| write size       | 79.3GB      |
| SST 파일 size      | 64MB        |
| Level-1 size     | 256MB       |
| Level multiplier | x10         |

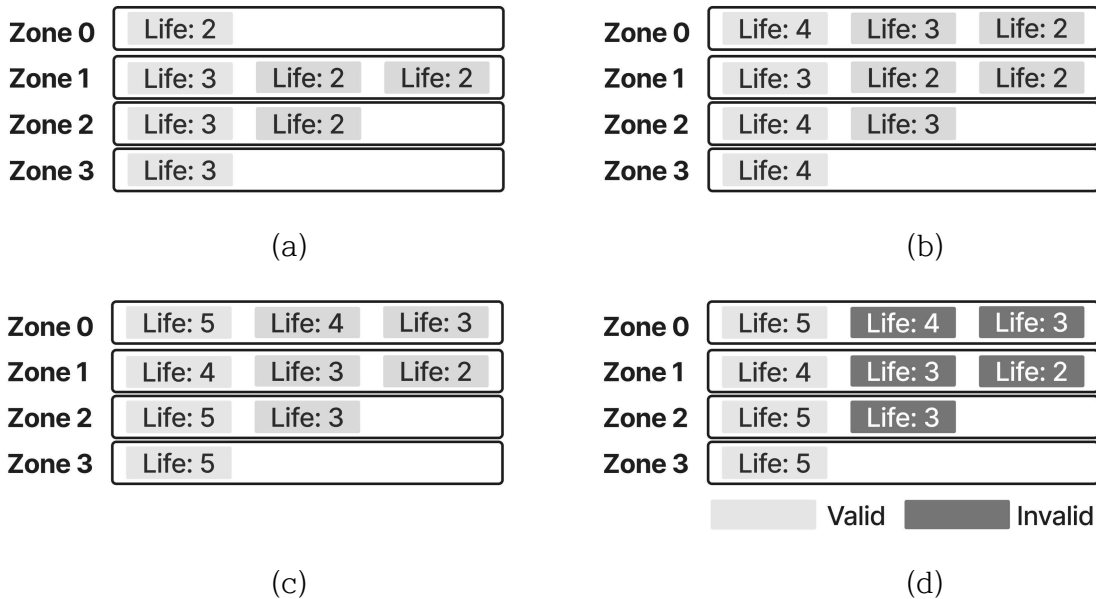
본 절에서는 RocksDB의 내장 벤치마크 툴인 *db\_bench*를 활용하여 현재 ZenFS의 zone 관리 정책들을 평가한다. 실험은 ZNS SSD를 에뮬레이트할 수 있는 가상 환경 FEMU에서 진행됐으며 자세한 구성은 [표 1]과 같다[21]. 그리고 *db\_bench*의 설정은 [표 2]와 같다[27]. 무작위 키 순서로 쓰기가 진행되며 총 102.4GB 용량의 ZNS SSD에 약 79.3GB의 데이터를 기록하였다. RocksDB는 기본적으로 Google의 snappy 알고리즘을 사용하여 SST 파일을 압축하며 압축 후 총 데이터는 약 40.3GB로 추정된다[30].

## 나. ZenFS의 zone 할당 정책 분석

[그림 8]은 *db\_bench* 수행 과정에서 ZNS SSD 내부의 사용 중인 zone과 전체 여유 공간(Free Space)의 비율 변화를 나타낸다. 실험 결과, ZNS SSD에 약 50% 수준의 충분한 여유 공간이 남아있음에도 불구하고 새로운 SST 파일을 쓰기 위한 zone 할당에 실패하는 현상이 관찰되었다. 이는 ZenFS의 zone 할당 정책이 실제 데이터 저장량보다 지나치게 많은 수의 빈 zone을 요구하기 때문이다.



[그림 8] *db\_bench* 수행 중 사용 중인 zone의 비율과 ZNS SSD의 여유 공간 비율



[그림 9] 워크로드 진행 정도에 따른 ZNS SSD의 내부 상태 (a)워크로드 초반

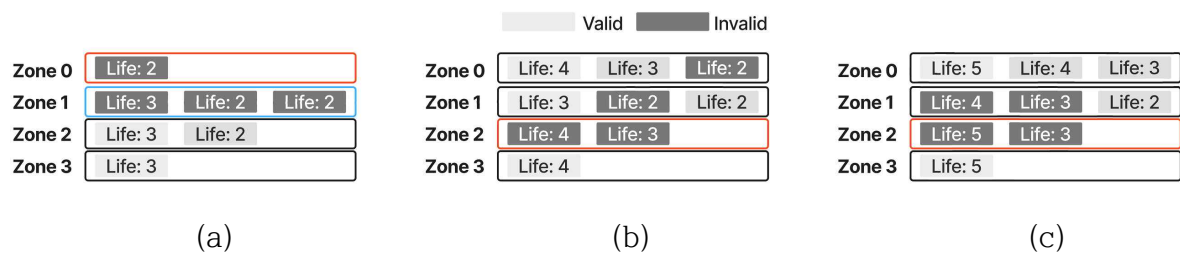
(b)워크로드 중반 (c)워크로드 후반 (d) 워크로드 후반 + 데이터 무효화

[그림 9]는 워크로드 진행 정도에 따른 ZNS SSD의 내부 상태를 도식화한 것이다. 앞서 II-3절에서 보았듯이 ZenFS는 zone에 처음 써진 SST 파일보다 짧은 수명의 SST

파일만 쓸 수 있고 그러한 zone이 없을 경우 새로운 zone을 할당한다. 다시 말해 ZenFS는 동일한 수명을 가진 SST 파일을 같은 zone에 배치하지 않고 항상 가장 긴 수명을 가진 SST 파일을 위해 새로운 zone을 할당한다. 특히 가장 긴 수명(Lifetime 5)을 가진 SST 파일이 생성되기 시작하면 해당 SST 파일을 쓰기 위해 항상 새로운 zone을 할당해야한다. 결과적으로 [그림 9-(c)]와 같이 모든 zone이 사용 중인 상태에서 수명이 5인 새로운 SST 파일 쓰기가 요청된다면 여유 공간은 충분하더라도 빈 zone이 고갈되어 zone 할당에 실패하게 된다. 뿐만 아니라 [그림 9-(a), (b)]의 상황에서도 모든 zone이 사용 중일 때 긴 수명의 SST 파일을 써야하는 상황이 발생한다면 zone 할당 실패로 이어진다.

[그림 9-(d)]는 [그림 9-(c)] 상황에서 데이터 무효화를 반영한 그림이다. [그림 9-(d)]의 zone 0, 1, 2와 같이 짧은 수명의 SST 파일들은 무효화되었지만 긴 수명의 SST 파일은 유효 상태로 남아있는 zone이 다수 발생한다. ZenFS는 이러한 정책을 이용해 가비지 컬렉션 시 유효 데이터 복사를 최소화하고자 했지만, 실제로는 전체 SST 파일이 무효화되기 전까지 해당 zone은 reset이 불가능하므로 빈 zone 확보를 방해하는 요인이 된다.

#### 다. ZenFS의 zone reset 정책 분석



[그림 10] 워크로드 진행 정도에 따른 zone reset의 발생 (a) 워크로드 초반 (b) 워크로드 중반 (c) 워크로드 후반

앞서 언급했듯이 ZenFS는 zone 할당 실패를 방지하고 사용 가능한 zone을 확보하기 위해 유효 데이터가 존재하지 않는 zone을 즉각적으로 reset하는 알고리즘을 채택하고 있다. 그러나 이 방식은 해당 zone의 실제 사용률을 고려하지 않는다는 한계가 있다. 즉 zone 내부의 사용 가능한 공간이 충분히 활용되지 않은 상태에서도 조기에 reset이 수행됨에 따라 불필요한 P/E cycle을 유발하고 이는 결과적으로 ZNS SSD의 수명에 부정적인 영향을 미친다.

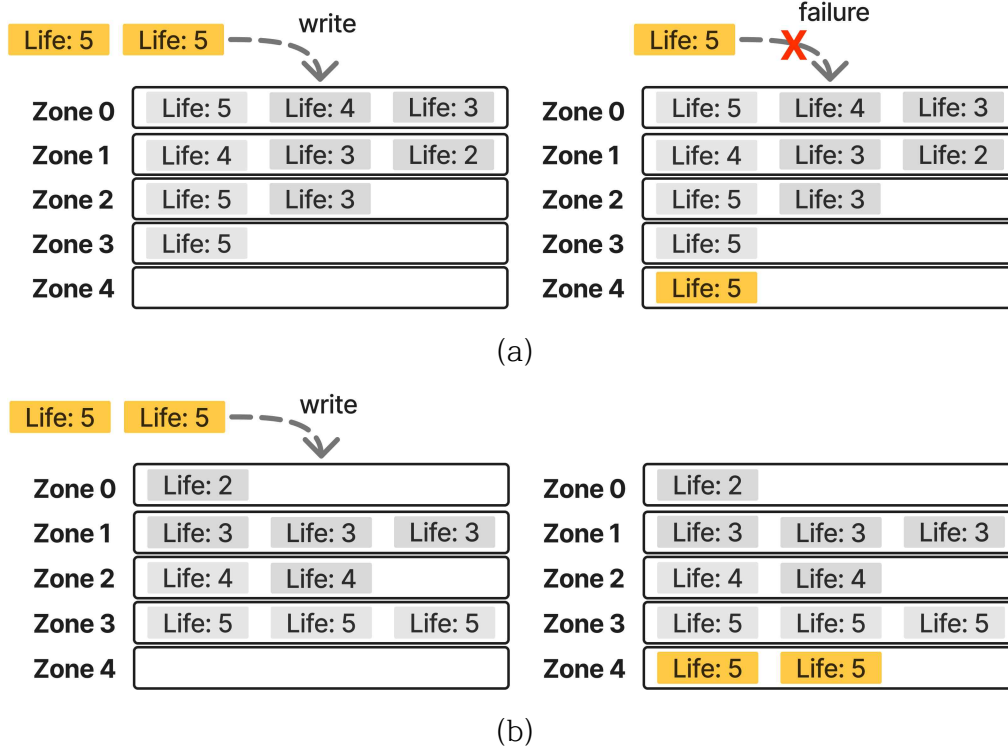


[그림 10]은 워크로드 수행 과정에서 발생하는 zone reset 양상을 시각화한 것이다. [그림 10-(a)]의 'Zone 1'과 같이 zone의 용량을 모두 사용한 상태에서 내부의 모든 SST 파일이 무효화되어 reset이 수행되는 경우, 이는 zone을 효율적으로 사용한 사례에 해당한다. 반면, [그림 10-(a)]의 'Zone 0'이나 [그림 10-(b)]의 'Zone 2', [그림 10-(c)]의 'Zone 2'와 같이 zone의 전체 용량을 채우지 못한 채 무효 데이터만 존재한다는 이유로 조기에 reset이 발생하는 경우, 이는 스토리지 자원의 활용도를 저해하는 비효율적인 reset으로 간주된다. 이처럼 불완전하게 사용된 zone에 대한 빈번한 reset 작업은 ZNS SSD의 한정된 P/E 사이클을 불필요하게 소모시켜 장치의 수명을 단축시킬 뿐만 아니라, 시스템 전반의 I/O 성능에도 부정적인 영향을 미친다.

III-1-가에서 논의한 실험에서 zone 할당 실패가 발생하지 않는 추가 실험 결과에 따르면 zone reset 알고리즘에 의해 reset된 zone의 약 60.2%가 70% 이상의 여유 공간을 보유하고 있었던 것으로 측정되었다. 이는 현 ZenFS의 zone reset 정책이 공간 활용 효율 측면에서 비효율적임을 보여준다.

## V. Strong Lifetime based Allocation(SLA)

### 1. SLA 정책의 개념 및 기대 효과



[그림 11] 각 정책별 ZNS SSD의 내부 상태 및 쓰기 요청 처리 결과 (a) ZenFS의 zone 할당 정책 (b) SLA

본 연구에서 제안하는 SLA는 동일한 수명을 가진 SST 파일들을 같은 zone에 배치하는 정책이다. 이는 수명별 데이터의 무효화 시점을 일치시켜 zone reset 효율을 극대화하고, 불필요한 빈 zone 소모를 억제하는 것을 목적으로 한다. [그림 11]은 기존 ZenFS와 SLA 정책의 데이터 배치 결과 및 쓰기 요청 처리 능력을 비교하여 보여준다.

수명이 5인 새로운 SST 파일에 대한 쓰기 요청이 두 차례 발생하는 상황을 가정할 때, 기존 ZenFS는 [그림 11-(a)]와 같이 첫 번째 요청에서 새로운 zone(Zone 4)을 할당해야 하며, 두 번째 요청 시에는 사용 가능한 빈 zone의 부재로 인해 쓰기 실패가 발생한다. 반면, SLA는 [그림 11-(b)]와 같이 첫 번째 요청에서 새로운 zone(Zone 4)을 할당한 후, 두 번째 SST 파일 또한 동일한 zone에 수용 가능하다. 이러한 방식은 ZNS SSD의 여유 공간 활용도를 높이고 새로운 zone 할당 빈도를 유의미하게 낮춘다.



없이 해당 zone에 SST 파일을 쓴다. 조건을 만족하는 zone이 존재하지 않는 경우 새로운 빈 zone을 할당한다. 이때 할당된 zone은 해당 SST 파일의 수명을 부여받아 이후 동일한 수명을 가진 SST 파일만 수용하는 zone으로 사용된다.

## VI. LZR(Lazy Zone Reset)

### 1. Lifetime별 SST 파일의 특징 분석

[표 3] SLA를 적용했을 때 reset된 zone의 사용률

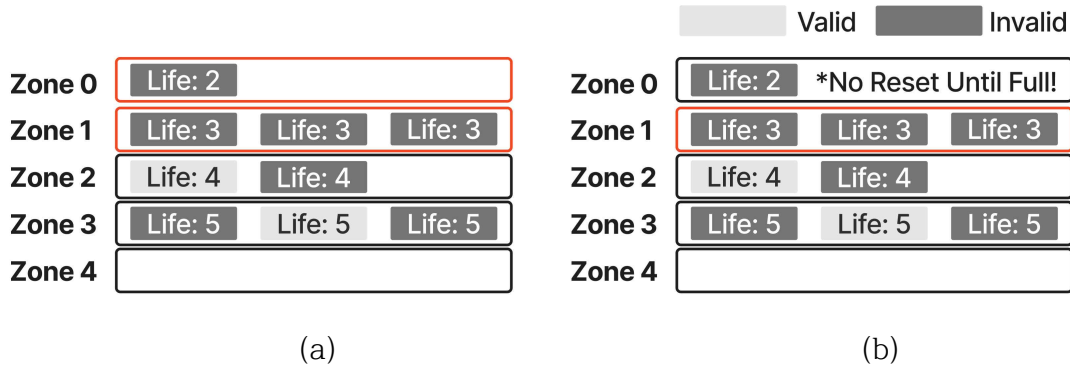
| Zone Lifetime                 | 2   | 3    | 4    | 5     |
|-------------------------------|-----|------|------|-------|
| Average Zone Utilization Rate | 12% | 100% | 100% | 98.5% |

본 연구에서 제안한 SLA 정책을 적용할 경우 각 zone은 수명에 따라 독립적으로 관리된다. [표 3]은 SLA 정책을 적용하여 벤치마크를 수행하였을 때 기존 zone reset 정책에 따라 실제 reset이 수행된 zone들의 사용률을 수명별로 분석한 결과이다. 분석 결과에 따르면, 수명이 3이상인 모든 zone은 데이터가 거의 가득 찬 상태에서 reset이 이루어짐으로써 높은 공간 효율성을 보여주었다. 반면, 수명 2를 부여받은 zone의 경우 사용률이 매우 낮은 상태에서 빈번하게 reset이 발생하는 현상이 관찰되었다. 이는 RocksDB의 데이터 기록 메커니즘과 밀접한 관련이 있다. II-2-나에서 보았듯, RocksDB는 안정성 확보를 위해 메모리 상의 MemTable과 저장 장치 상의 로그 파일([그림 2]의 WAL)을 유지한다. 해당 로그 파일은 MemTable의 데이터가 ZNS SSD로 안전하게 플러시되는 즉시 삭제되는 짧은 수명 특성을 지닌다.

SLA 정책 하에서 이러한 로그 파일들은 수명 2인 zone에 배치되는데, 파일의 생성과 삭제 주기가 매우 짧아 zone의 공간을 충분히 채우기도 전에 reset 조건(유효 데이터 없이 무효 데이터만 존재)을 만족하게 된다. 결과적으로 수명이 2인 zone에서는 극히 일부의 공간만 사용된 채 reset이 반복되어, ZNS SSD의 전체적인 reset 횟수를 불필요하게 증가시키는 원인이 된다.

### 2. LZR 정책의 개념 및 동작 원리

앞서 언급한 과도한 zone reset 문제를 해결하기 위해, 본 논문에서는 짧은 수명 zone에 특화된 LZR 정책을 제안한다. LZR의 핵심 아이디어는 수명이 2인 zone에 유효 데이터가 존재하지 않더라도, 해당 zone이 완전히 채워지지 않았다면 reset을 미뤄 공간 활용도를 극대화하는 것이다.



[그림 14] SLA와 함께 적용한 zone reset 정책

[그림 14]는 기존 ZenFS의 reset 방식과 제안하는 LZR 정책의 차이점을 도식화하여 비교한다. 기존 ZenFS([그림 14-(a)])의 경우 zone 내의 데이터가 모두 무효화되면 zone의 사용률과 관계없이 즉시 zone을 reset하여 사용 가능한 zone을 확보한다. 반면 LZR([그림 14-(b)])은 'Zone 0'과 같이 무효 데이터만 존재하는 zone이라도 zone의 수명이 2이고 가득 차지 않았다면 reset 대상에서 제외시킨다. 이를 통해 해당 zone의 남은 공간에 로그 데이터를 계속해서 쓸 수 있게 함으로써 reset 빈도를 획기적으로 낮춘다. 이는 자주 써지고 빠르게 삭제되는 로그 파일의 특성을 고려한 설계이다.

### 3. LZR 알고리즘 구현

LZR 정책의 구체적인 구현 로직을 [그림 15]의 수도 코드로 작성하였다. 본 알고리즘은 RocksDB에서 SST 파일 또는 로그 파일이 삭제되어 zone reset이 필요할 때마다 호출된다.

---

#### Algorithm 3 LZR

---

```

1: for all  $z \in \text{io\_zones}$  do
2:   if  $z.\text{lifetime} == 2$  and  $z.\text{capacity} \neq 0$  then
3:     continue
4:   end if
5:   if  $z.\text{wp} \neq z.\text{start}$  and  $z.\text{used\_capacity} == 0$  then
6:     ResetZone( $z$ )
7:   end if
8: end for

```

---

[그림 15] LZR 수도 코드

알고리즘의 주요 수행 단계는 다음과 같다. 먼저 ZNS SSD 내의 모든 zone을 스캔하여 유효 데이터가 존재하지 않는 zone을 탐색한다. 유효 데이터가 없는 zone을 발견

하면 해당 zone의 수명이 2인지 확인한다. 해당 zone의 수명이 2가 아닌 경우(일반 SST 파일 zone) 즉시 reset을 수행한다. 해당 zone의 수명이 2인 경우 zone이 가득 찬 상태인지 추가로 검사하고 가득 찬 경우에만 reset을 수행한다.

이러한 조건부 zone reset 메커니즘을 통해 LZR은 로그 파일의 잦은 갱신으로 인한 ZNS SSD의 마모를 방지하고, 장치의 신뢰성 및 기대 수명을 보호하는 데 기여한다.

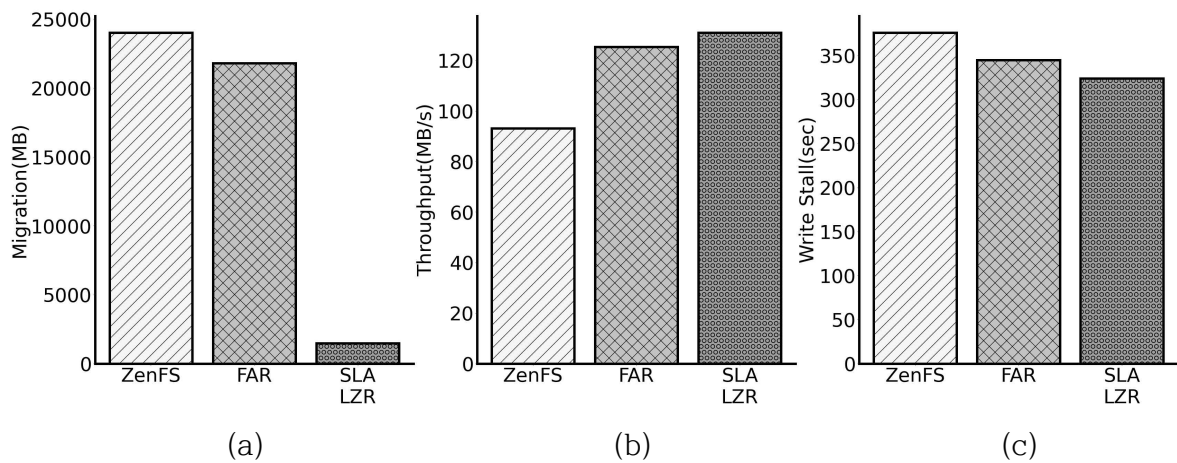
## VII. 실험 및 평가

### 1. 실험 환경

실험은 III장에서 수행한 실험과 같은 환경에서 진행하였다([표 1] 참조). 벤치마크 프로그램은 RocksDB의 내장 벤치마크 툴인 *db\_bench*의 무작위 키 입력 워크로드인 fillrandom과 실제 데이터 센터 운영 환경과 비슷한 키 접근 및 밸류 크기를 사용하는 워크로드인 mixgraph를 사용하였다. 그리고 실제 클라우드 서비스 환경의 접근을 모사한 NoSQL 및 클라우드 데이터 저장소의 성능 평가 도구로 널리 알려진 YCSB(Yahoo! Cloud Serving Benchmark)를 사용하였다[27, 28, 29].

### 2. *db\_bench* fillrandom 실험

#### 가. 성능 관련 지표 분석



[그림 16] (a) 데이터 복사량 비교 (b) 처리량 비교 (c) 쓰기 중단 발생 시간 비교

*db\_bench*의 옵션은 [표 2]와 같고 ZenFS의 경우 zone 할당 실패가 발생하지 않은 상태의 결과를 사용했다. FAR과 비교를 위해 FAR이 구현된 RocksDB v7.4를 사용했으며, FAR은 ZenFS v2.1에서 자체 가비지 컬렉션 정책을 구현하였다. ZenFS와 SLA+LZR은 최신 ZenFS 버전과 ZenFS에서 제공하는 가비지 컬렉션 정책을 사용했다. 결과는 벤치마크를 10번 수행하여 얻은 평균값이다.

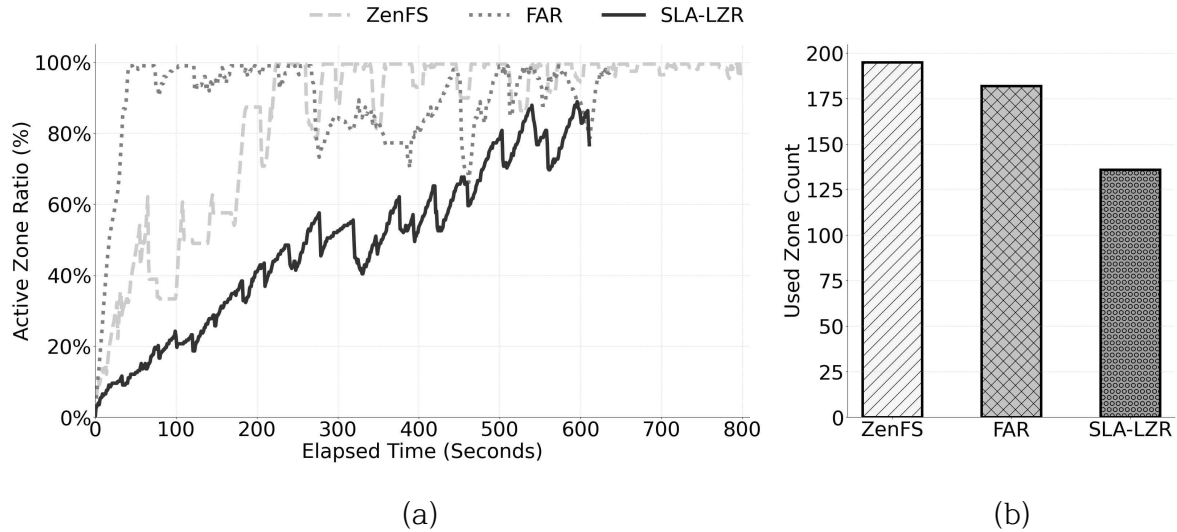
본 연구에서 제안한 SLA-LZR 정책의 성능을 검증하기 위해 가비지 컬렉션 발생 시



수반되는 데이터 복사(Migration)량, 시스템 전체의 처리량(Throughput) 및 쓰기 중단 발생 시간을 측정하였다.

[그림 16-(a)], [그림 16-(b)]는 각각 데이터 복사량과 처리량을 비교한다. 분석 결과, SLA-LZR은 개선된 zone 할당 및 reset 메커니즘을 통해 유효 데이터의 격리 효율을 높임으로써, 기존 ZenFS 정책 대비 가비지 컬렉션 유발 빈도를 획기적으로 낮추었다. 이를 통해 가비지 컬렉션 과정에서 발생하는 데이터 복사량을 기존 대비 약 93.5% 감소시키는 성과를 거두었다. 이러한 I/O 부하의 감소는 즉각적인 성능 향상으로 이어져, 시스템 처리량 측면에서 기존 ZenFS 대비 약 40.6%, FAR 대비 약 4.5% 향상된 결과를 보여주었다. FAR 알고리즘은 할당 가능한 zone이 고갈될 경우, zone과 SST 파일의 수명과 무관하게 임의의 zone에 SST 파일을 쓴다. 이러한 정책은 단기적으로 사용 가능한 zone 부족을 방지하여 ZenFS 대비 향상된 처리량을 확보하는 데 기여한다. 그러나 장기적인 운용 관점에서는 단일 zone 내에 서로 다른 수명의 SST 파일들이 혼재됨에 따라 가비지 컬렉션 수행 시 유효 데이터 복사량을 증가시킬 수 있다. 위 실험 결과는 수명 기반의 정밀한 zone 관리가 저장 장치 대역폭 낭비를 방지하는 데 결정적인 역할을 수행함을 보여준다.

RocksDB 운영 환경에서 발생하는 쓰기 지연 현상은 시스템의 응답성을 저해하는 주요 요인이다. 일반적으로 쓰기 지연은 MemTable의 플러시 대기, L0 SST 파일의 누적, 또는 백그라운드 컴팩션 부하가 임계치를 초과할 때 발생하며, 이는 모두 저장 장치의 I/O 성능과 직결된다. [그림 16-(c)]는 각 정책별 쓰기 지연 발생 빈도를 측정한 결과이다. SLA-LZR은 가비지 컬렉션에 따른 I/O 블로킹(Blocking) 현상을 최소화함으로써, RocksDB가 내부 작업을 원활히 수행할 수 있는 I/O 자원을 확보하였다. 그 결과, ZenFS 대비 약 13.8%, FAR 대비 약 6.1%의 쓰기 지연 감소 효과를 달성하였다. 이는 제안 기법이 단순히 처리량만을 높이는 것이 아니라, 사용자 요청에 대한 서비스 안정성을 확보하는 데에도 기여하고 있음을 의미한다.



[그림 17] (a) 실행 시간에 따른 사용 zone 비율 (b) 벤치마크 종료 후 사용 zone 수

#### 나. 공간 활용 효율성 평가

마지막으로 벤치마크 수행 과정 및 종료 시점에서의 ZNS SSD 공간 활용 효율성을 평가한다. [그림 17-(a)]는 벤치마크 실행 시간에 따른 사용 zone의 비율을 나타내고, [그림 17-(b)]는 벤치마크 종료 시점에 사용 중인 zone의 수를 나타낸다.

[그림 17-(a)]를 살펴보면 워크로드 초기 단계에서 기존 ZenFS와 FAR은 사용 중인 zone의 비율이 가파르게 증가하는 모습을 보인다. 특히 FAR의 증가 폭이 더욱 큰 것을 볼 수 있는데, 이는 ZNS SSD의 사용 가능한 용량이 충분한 초기 상태에서 reset 임계값(Twp)가 높게 설정되어 zone reset을 지연시키기 때문이다. 그러나 사용 중인 zone의 비율이 100%에 근접함에 따라, FAR의 zone 할당 정책은 수명을 고려하지 않은 채 SST 파일들을 한 zone에 모으게 된다. 이는 빠른 가비지 컬렉션을 유발하는 원인이 되며, 실제 그래프 상에서 사용 중인 zone의 비율이 급격히 하락하는 지점은 가비지 컬렉션을 통한 강제적인 zone 회수가 발생했음을 시사한다. 반면 제안하는 SLA-LZR은 가장 완만한 증가를 유지하며 안정적인 모습을 보였다. 이는 SLA-LZR이 무분별한 새로운 zone 할당을 억제하고 수명 인지 적응형 reset 정책을 통해 사용 가능한 zone을 적시에 확보함을 보여준다.

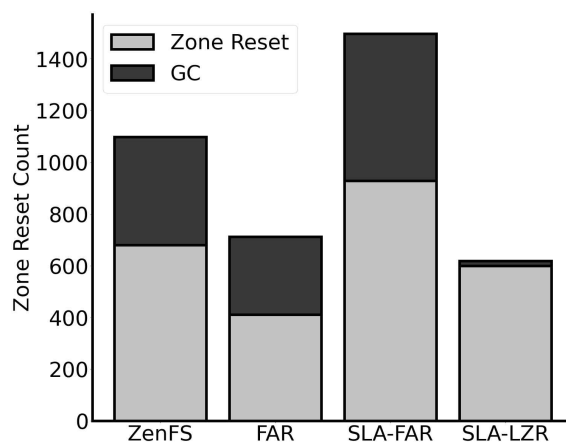
벤치마크 종료 후, SLA-LZR은 ZenFS 및 FAR 대비 약 27.9% 더 적은 수의 zone을 사용하면서도 동일한 워크로드를 성공적으로 완수하였다. 이는 본 연구에서 제안한 정책이 ZNS SSD의 물리적 자원을 보다 밀도 있게 활용하고 있음을 보여준다. 특히 물

리적 공간이 제한적인 환경에서도 zone이 고갈되는 현상 없이 안정적인 시스템 운영이 가능함을 시사하며, 이는 차세대 스토리지 시스템의 공간 관리 효율성을 극대화하는 핵심 정책으로 작용할 수 있다.

#### 다. 저장 장치 신뢰성 및 zone reset 효율성 분석

ZNS SSD의 수명 연장 및 물리적 마모 최소화 성능을 평가하기 위해, 각 정책별 zone reset 횟수를 비교 분석하였다. 실험 환경 및 *db\_bench* 설정은 앞의 실험과 동일하며, 총 reset 횟수는 'reset 정책에 의한 reset'과 '가비지 컬렉션에 의한 reset'의 합계로 산출하였다.

[그림 18]은 ZenFS, FAR, SLA-FAR 및 본 연구에서 제안한 SLA-LZR의 zone reset 횟수를 보여준다. 분석 결과, 각 데이터의 수명 특성을 반영하여 차등적인 reset 메커니즘을 적용한 SLA-LZR이 가장 낮은 reset 빈도를 기록하며 최적의 효율성을 입증하였다.



[그림 18] zone reset 횟수 비교

기존의 FAR 정책은 여유 공간 기반의 임계치를 적용하여 reset 횟수는 일부 감소시켰으나, 앞서 분석한 바와 같이 수명이 다른 데이터의 혼재로 인한 가비지 컬렉션 유발 빈도가 높아 결과적으로 GC에 의한 reset 횟수가 크게 증가하는 한계를 보였다.

[표 3]의 데이터에 근거할 때, SLA 정책 하에서 수명 힌트가 3, 4, 5인 zone들은 대부분 데이터가 가득 찬(Full) 상태에서 reset되므로, FAR의 여유 공간 기반 정책이 해당 zone들의 reset 빈도를 낮추는 데 기여하는 바는 미미하다. 반면, 수명 2를 부여받은 zone은 RocksDB 로그 파일(WAL)의 특성상 파일의 생성과 삭제가 빈번하며, zone의

일부 공간만 점유한 채 삭제되는 패턴을 반복한다. SLA-FAR의 경우, 이러한 짧은 수명 파일의 특성을 고려하지 않고 일괄적인 reset 정책을 적용함에 따라 불필요한 reset 작업이 빈번하게 수행되는 결과를 초래하였다.

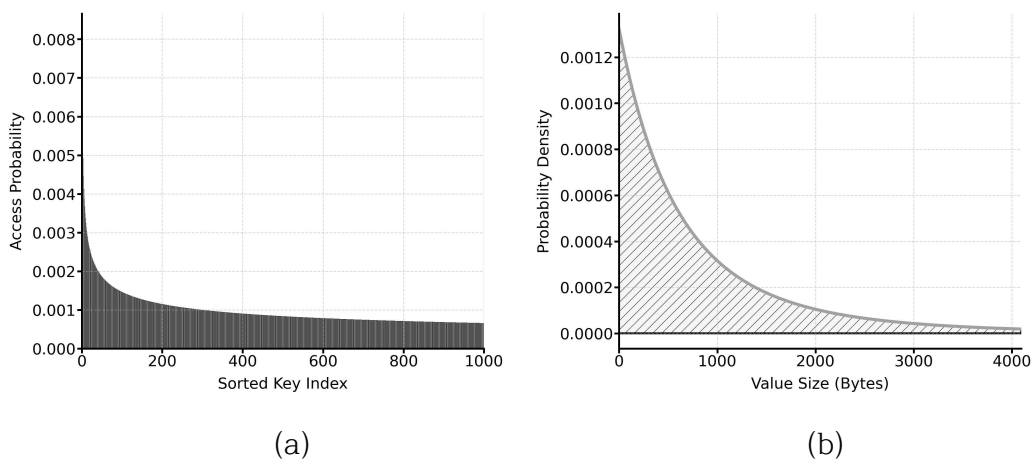
이와 대조적으로 SLA-LZR은 짧은 수명을 가진 zone(수명 2)에 대해 zone이 완전히 채워질 때까지 reset을 미루는 전용 정책을 시행함으로써, 로그 파일의 잦은 갱신에도 불구하고 안정적인 zone 관리를 실현하였다. 결과적으로 SLA-LZR은 기존 ZenFS 대비 전체 zone reset 횟수를 약 43.7% 감소시켰고, 이는 ZNS SSD의 물리적 마모도를 획기적으로 낮추어 저장 장치의 기대 수명을 연장할 수 있는 실질적인 방안을 시사한다.

### 3. *db\_bench* mixgraph 실험

#### 가. 실험 워크로드 개요

RocksDB는 실제 데이터센터의 운영 환경과 유사한 벤치마크 환경을 제공하기 위해 *db\_bench* 내에 mixgraph 워크로드를 지원한다[7]. 해당 워크로드의 기반이 된 선행 연구에 따르면, 실제 키-밸류 데이터베이스 트레이스를 분석한 결과 기존의 예측과 달리 키의 크기가 시스템 성능에 영향을 미칠 만큼 비대해지는 경우가 존재하며, 전체 키 공간 중 매우 적은 특정 범위의 키에 접근 집중되는 핫스팟(Hotspot) 현상이 빈번하게 발생한다고 분석했다. 본 실험에서는 이러한 실측 데이터를 기반으로 수학적으로 모델링된 키-밸류 분포를 적용하여 현실적인 워크로드를 재현하였다.

#### 나. 실험 환경 설정

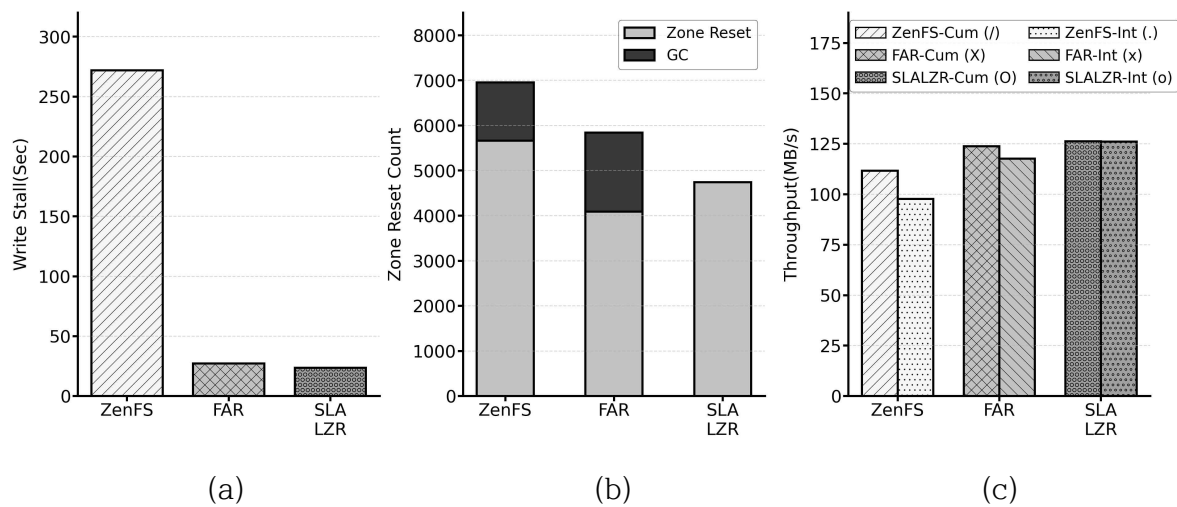


[그림 19] mixgraph의 (a) 각 키에 접근될 확률 (b) 밸류 크기 분포

본 실험에서는 키 접근 빈도와 밸류 크기 분포에 모두 파레토(Pareto) 계열의 불균형 분포를 적용하였다. 키 분포의 경우, 전역적으로 균일한 무작위 접근이 아닌 특정 키 범위(Key-range)에 요청이 집중되는 핫스팟 특성을 모사하였다. 밸류 크기 분포의 경우, 1KB 미만의 소형 밸류가 압도적인 비중을 차지하는 긴 꼬리(Long-tail) 형태의 일반 파레토 분포를 적용하였으며, 최대 밸류 크기는 4KB로 제한하였다. [그림 19-(a)와 (b)]는 각각 키 접근 확률과 밸류 크기 분포를 나타낸 그래프이다.

그리고 원활한 실험을 위해 기존 fillrandom 실험의 SST 파일 크기 64MB를 128MB로 확장하여 실험을 진행하였다. 이러한 큰 SST 파일 설정은 LSM-Tree의 레벨 별 SST 파일 수를 감소시킴으로써 빈 zone의 고갈로 인한 zone 할당 실패의 가능성을 낮춘다.

#### 다. 실험 결과



[그림 20] (a) 쓰기 중단 시간 비교 (b) zone reset 횟수 비교 (c) 처리량 비교

[그림 20]은 각 정책별 쓰기 중단 시간, zone reset 횟수 그리고 누적(Cumulative) 처리량과 간격(Interval) 처리량의 차이를 비교한 결과이다. 우선 [그림 20-(a)] 쓰기 중단 시간의 경우 fillrandom 실험 결과인 [그림 16-(c)]와 비교하였을 때 전체적으로 크게 감소하였다. 이는 앞서 설명한 바와 같이 SST 파일의 크기를 확장함에 따라 zone 할당 실패를 줄임으로써 발생하는 결과이다.

다음으로 [그림 20-(b)] zone reset 횟수를 살펴보면 zone에 무효 데이터만 존재하여 발생하는 zone reset 횟수는 [그림 18]의 fillrandom때와 유사하지만, 가비지 컬렉

선에 의해 발생한 zone reset 횟수는 FAR이 ZenFS보다 높아졌다. 이는 mixgraph 워크로드의 극단적인 핫 키 집중 쓰기 특성과 밀접한 관련이 있다. 핫 키들이 지속해서 업데이트 됨에 따라 키 범위가 겹치는 여러 SST 파일이 한 번에 컴팩션 되고, 이는 zone 내의 무효 데이터의 수를 늘리게 된다. FAR의 경우 새로운 zone이 부족해지면 SST 파일의 수명을 고려하지 않고 여유 공간이 충분한 zone에 무작위로 데이터를 배치하므로, 유효 데이터와 무효 데이터가 혼재되어 zone reset을 불가능하게 만들고 가비지 컬렉션을 가속하게 된다. 반면 본 논문에서 제안하는 SLA-LZR은 수명에 기반한 데이터 배치와 수명에 따른 적절한 reset 정책을 통해 zone을 적시에 확보하여 가비지 컬렉션이 발생하지 않은 모습을 보여준다.

마지막으로 [그림 20-(c)]는 각 정책의 누적 처리량과 간격 처리량을 비교하였다. 간격 처리량은 종료 직전 1초간 측정된 처리량이다. ZenFS의 경우 쓰기 중단과 가비지 컬렉션으로 인해 간격 처리량이 누적 평균 대비 약 12.5% 감소하는 전형적인 꼬리 지연(Tail latency) 현상을 보였다. FAR 역시 가비지 컬렉션의 영향으로 간격 처리량이 약 5% 감소하였다. 반면 SLA-LZR은 쓰기 지연 시간을 최소화하고 가비지 컬렉션을 완벽히 억제함으로써 성능 편차가 거의 존재하지 않았으며, 누적 처리량과 간격 처리량이 거의 일치하는 시스템 안정성을 보였다.

## 4. YCSB(Yahoo! Cloud Serving Benchmark) 실험

### 가. YCSB 실험 구성 및 워크로드 특성

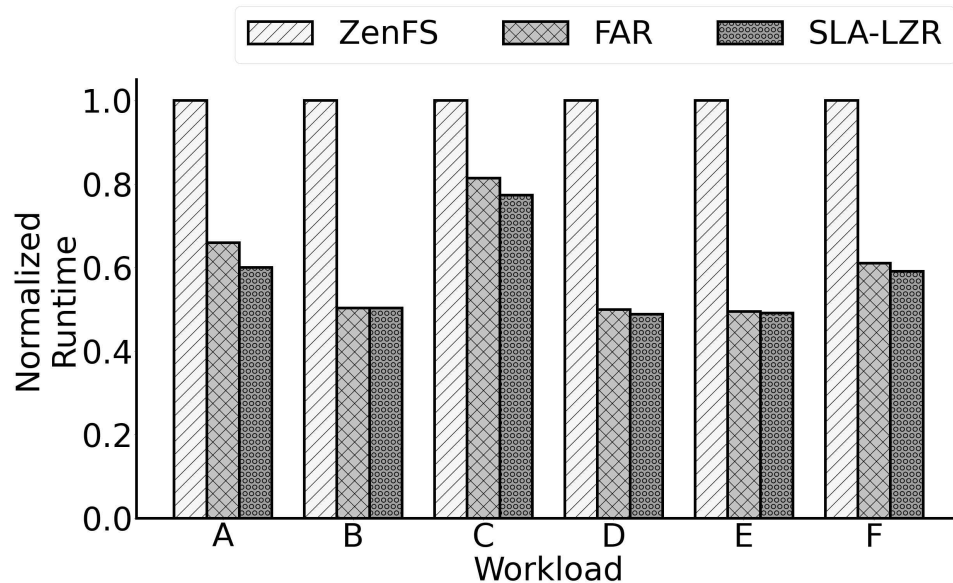
[표 4] YCSB 워크로드 특성

| Workload     | A       | B       | C       | D      | E             | F              |
|--------------|---------|---------|---------|--------|---------------|----------------|
| R:W:U        | 1:0:1   | 95:0:5  | 1:0:0   | 95:5:0 | 95:5<br>(S:W) | 1:1<br>(R:R-W) |
| Distribution | zipfian | zipfian | zipfian | latest | zipfian       | zipfian        |

제안한 정책의 실효성을 다각도로 검증하기 위해, 실제 클라우드 서비스 환경의 접근 패턴을 모사한 YCSB 벤치마크를 수행하였다. [표 4]는 실험에 사용된 각 워크로드별 읽기(Read), 쓰기(Insert), 업데이트(Update)의 구성 비율 및 세부 특성을 나타낸다. 워크로드 E는 데이터 베이스 전체 스캔(Scan)의 비중이 높으며, 워크로드 F는 읽기-수정-쓰기(Read-modify-write) 패턴을 가진다.

본 실험에서는 가비지 컬렉션이 시스템 성능에 미치는 영향을 명확히 분석하기 위해, 모든 정책에 대해 가비지 컬렉션 임계치 직전까지 데이터를 사전에 기록한 후 각 작업을 수행하였다. 성능 지표로는 워크로드 완료까지 소요된 전체 실행 시간(Execution Time)을 측정하였다.

#### 나. 실행 시간 분석 및 가비지 컬렉션의 영향



[그림 21] YCSB 실행 시간

[그림 21]은 기존 ZenFS의 실행 시간을 기준으로 각 정책의 성능을 정규화(Normalized)한 결과이다.

실험 결과, 읽기 전용 워크로드인 Workload C의 경우 추가적인 데이터 기록에 따른 GC가 발생하지 않으므로 모든 정책에서 실행 시간의 유의미한 차이가 관찰되지 않았다. 그러나 쓰기 작업이 포함된 나머지 모든 워크로드(A, B, D, E, F)에서 SLA-LZR은 기존 정책 대비 압도적인 성능 향상을 기록하였다.

구체적으로 SLA-LZR은 ZenFS 대비 실행 시간을 워크로드 A에서 40.0%, B에서 49.7%, D에서 51.14%, E에서 50.9%, 그리고 F에서 40.8% 단축시키는 성과를 거두었다. 이를 통해 최신 연구인 FAR 대비 약 0.01% ~ 9.0% 실행 시간을 단축 시켰다. FAR와의 성능 차이는 워크로드 A와 F에서 가장 잘 드러나는데, 이는 타 워크로드 대비

쓰기 비중이 높아 zone 할당 정책 및 zone reset 정책의 개선을 가장 잘 보여줄 수 있는 워크로드이기 때문이다. FAR은 ZenFS보다 적은 가비지 컬렉션을 발생시켰으며 이러한 결과는 V-2.가절의 실험 분석과 마찬가지로 FAR 정책이 할당 가능한 zone이 고갈된 경우 SST 파일과 zone의 수명을 고려하지 않고 임의의 zone에 SST 파일을 쓰는 배치 정책을 사용하기 때문이다. 이러한 정책은 단기적으로는 zone 할당 실패에 따른 I/O 블로킹을 방어해 throughput 향상을 보일 수 있으나, 본 연구에서 지정한 바와 같이 장기적인 관리 효율성 측면에서는 한계를 지닌다.

ZenFS와 비교했을 때 성능 차이의 근본적인 원인은 가비지 컬렉션 발생에 따른 I/O 블로킹 유무에 기인한다. 분석 결과, 기존 ZenFS는 쓰기 작업이 포함된 워크로드 수행 중 빈번한 가비지 컬렉션을 발생시켰으며, 이 과정에서 수반되는 데이터 복사 부하가 사용자 요청 처리 대역폭을 낮춰 실행 시간을 지연시켰다.

반면, SLA-LZR은 본 연구에서 제안한 수명 기반 zone 할당 및 수명별 적응형 reset 정책을 통해 사용 가능한 zone을 최적으로 관리함으로써, 모든 YCSB 워크로드 수행 과정에서 단 한 차례의 가비지 컬렉션도 유발하지 않았다. 결과적으로 가비지 컬렉션으로 인한 I/O 부하를 원천적으로 제거함으로써 애플리케이션 계층에 온전한 스토리지 대역폭을 제공할 수 있었으며, 이는 실제 서비스 환경에서 제안 기법이 높은 응답성과 처리 성능을 보장할 수 있음을 입증한다.



## VIII. 결론

본 논문에서는 ZNS SSD 환경에서 RocksDB 기반의 사용자 레벨 파일시스템인 ZenFS가 직면한 zone 관리 효율성 및 장치 신뢰성 한계를 분석하고, 이를 해결하기 위한 데이터 수명 기반 zone 할당 및 reset 정책인 SLA-LZR을 제안하였다.

실험 분석 결과, 기존 ZenFS의 zone 할당 정책은 물리적 공간이 충분함에도 불구하고 사용 가능한 zone이 조기에 고갈되는 zone 할당 실패(Zone Allocation Failure) 현상을 야기함을 확인하였다. 또한, 최신 연구인 FAR 정책은 zone이 부족할 시 데이터 혼재를 허용함으로써 단기적인 성능 향상을 보였으나, 장기적으로는 가비지 컬렉션 시의 데이터 복사 오버헤드를 가중시키는 한계를 보였다.

본 연구에서 제안한 SLA 정책은 동일한 수명을 가진 SST 파일을 같은 zone에 배치함으로써 데이터 무효화 시점을 일치시키고 zone reset 효율을 극대화하였다. 이와 더불어 제안된 LZR 정책은 RocksDB 로그 파일의 특성을 고려하여, 짧은 수명 zone의 reset 시점을 최적화함으로써 불필요한 장치 마모를 방지하였다.

이러한 정책을 바탕으로 SLA-LZR은 가비지 컬렉션 발생 빈도를 획기적으로 낮추어 데이터 복사량을 기존 대비 약 93.5% 감소시켰다. 이를 통해 전체 시스템 처리량을 ZenFS 대비 40.6%, FAR 대비 4.5% 향상시켰다. 그리고 가비지 컬렉션에 따른 I/O 블로킹을 최소화함으로써 RocksDB의 쓰기 지연 현상을 ZenFS 대비 약 13.8% 완화하여 실제 서비스 환경에서의 응답성을 개선하였다. 또한, 수명별 특성에 최적화된 reset 메커니즘을 통해 불필요한 zone reset 횟수를 ZenFS 대비 43.7% 감소시켰으며, 이는 ZNS SSD의 물리적 수명 연장에 기여할 수 있음을 입증하였다. 추가적으로 실제 워크로드와 유사한 환경인 YCSB 벤치마크 평가 결과, 읽기/쓰기 혼합 워크로드 전반에서 ZenFS 대비 최대 51.14%의 실행 시간 단축 효과를 거두었으며, 특히 모든 실험 과정에서 가비지 컬렉션을 발생시키지 않는 최적의 자원 관리 능력을 보였다.

결론적으로 본 논문이 제안한 SLA-LZR 정책은 차세대 스토리지 인터페이스인 ZNS 환경에서 애플리케이션의 데이터 특성을 소프트웨어 계층의 관리 정책에 효과적으로 반영할 수 있음을 보여주었다. 본 연구의 결과는 고성능·저지연이 요구되는 데이터센터 및 클라우드 스토리지 시스템의 효율성을 극대화하는 핵심 기술로 활용될 것으로 기대된다.

## 참고문헌

- [1] J. Han, E. Haihong, G. Le, and J. Du, "Survey on NoSQL database," in Proc. 6th Int. Conf. Pervasive Comput. Appl. (ICPCA), Port Elizabeth, South Africa, 2011, pp. 363-366.
- [2] T. Chen et al., "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," arXiv preprint arXiv:1512.01274, 2015.
- [3] C. Lai et al., "Atlas: Baidu's key-value storage system for cloud data," in Proc. 31st Symp. Mass Storage Syst. Technol. (MSST), Santa Clara, CA, USA, 2015, pp. 1-14.
- [4] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications," ACM Trans. Storage, vol. 17, no. 4, pp. 1-32, 2021.
- [5] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," Acta Inform., vol. 33, no. 4, pp. 351-385, 1996.
- [6] C. Luo and M. J. Carey, "LSM-based storage techniques: a survey," VLDB J., vol. 29, no. 1, pp. 393-418, 2020.
- [7] Z. Cao, S. Dong, S. Vemuri, and D. H. Du, "Characterizing, modeling, and benchmarking RocksDB Key-Value workloads at facebook," in Proc. 18th USENIX Conf. File Storage Technol. (FAST 20), Santa Clara, CA, USA, 2020, pp. 209-223.
- [8] M. Rosenblum and J. K. Ousterhout, "The design and implementation of a log-structured file system," ACM Trans. Comput. Syst., vol. 10, no. 1, pp. 26-52, 1992.
- [9] H. Chen, C. Ruan, C. Li, X. Ma, and Y. Xu, "SpanDB: A fast, Cost-Effective LSM-tree based KV store on hybrid storage," in Proc. 19th USENIX Conf. File Storage Technol. (FAST 21), Santa Clara, CA, USA, 2021, pp. 17-32.
- [10] W. Zhong, C. Chen, X. Wu, and S. Jiang, "REMIX: Efficient range query for LSM-trees," in Proc. 19th USENIX Conf. File Storage Technol. (FAST 21), Santa Clara, CA, USA, 2021, pp. 51-64.

- [11] H. Sun, S. Dai, and J. Huang, "Exploring cascaded write amplification in lsm-tree based key-value stores with solid-state disks," in Proc. IEEE 21st Int. Conf. High Perform. Comput. Commun. (HPCC), Zhangjiajie, China, 2019, pp. 2072-2077.
- [12] M. Bjørling et al., "ZNS: Avoiding the block interface tax for flash-based SSDs," in Proc. USENIX Annu. Tech. Conf. (USENIX ATC 21), 2021, pp. 689-703.
- [13] M. Bjørling, "From open-channel SSDs to zoned namespaces," in Proc. Linux Storage Filesystem Conf. (Vault), Boston, MA, USA, vol. 1, 2019, p. 20.
- [14] T. Stavrinou, D. S. Berger, E. Katz-Basnett, and W. Lloyd, "Don't be a blockhead: zoned namespaces make work on conventional SSDs obsolete," in Proc. Workshop Hot Topics Operating Syst. (HotOS), Ann Arbor, MI, USA, 2021, pp. 144-151.
- [15] H. Wang et al., "Efficient data placement for zoned namespaces (ZNS) ssds," in Proc. IFIP Int. Conf. Network Parallel Comput. (NPC), Jinan, China, 2022, pp. 302-314.
- [16] P. Jin, X. Zhuang, Y. Luo, and M. Lu, "Exploring index structures for zoned namespaces SSDs," in Proc. IEEE Int. Conf. Big Data, Orlando, FL, USA, 2021, pp. 5919-5922.
- [17] D. Purandare, P. Wilcox, H. Litz, and S. Finkelstein, "Append is near: Log-based data management on ZNS SSDs," in Proc. 12th Annu. Conf. Innovative Data Syst. Res. (CIDR), Chaminade, CA, USA, 2022, pp. 1-8.
- [18] Western Digital Corporation, "ZenFS: RocksDB Storage Backend for ZNS SSDs and SMR HDDs," 2020. [Online]. Available: <https://github.com/westerndigitalcorporation/zenfs>
- [19] H. Holmberg, "ZenFS, zones and RocksDB—Who likes to take out the garbage anyway," in Proc. Storage Develop. Conf. (SDC), Santa Clara, CA, USA, 2020, pp. 1-12.
- [20] S. Byeon, J. Ro, S. Jamil, J. U. Kang, and Y. Kim, "A free-space adaptive runtime zone-reset algorithm for enhanced ZNS efficiency," in Proc. 15th ACM Workshop Hot Topics Storage File Syst. (HotStorage), Boston, MA, USA, 2023, pp. 109-115.

- [21] H. Li, M. Hao, M. H. Tong, S. Sundararaman, M. Bjørling, and H. S. Gunawi, "The CASE of FEMU: Cheap, accurate, scalable and extensible flash emulator," in Proc. 16th USENIX Conf. File Storage Technol. (FAST 18), Oakland, CA, USA, 2018, pp. 83-90.
- [22] L. M. Grupp, J. D. Davis, and S. Swanson, "The bleak future of NAND flash memory," in Proc. 10th USENIX Conf. File Storage Technol. (FAST 12), San Jose, CA, USA, vol. 7, no. 3.2, 2012, pp. 10-12.
- [23] R. Verschoren and B. Van Houdt, "On the impact of garbage collection on flash-based SSD endurance," in Proc. 4th Workshop Interactions of NVM/Flash with Operating Syst. Workloads (INFLOW 16), Savannah, GA, USA, 2016, pp. 1-8.
- [24] B. Van Houdt, "A mean field model for a class of garbage collection algorithms in flash-based solid state drives," ACM SIGMETRICS Perform. Eval. Rev., vol. 41, no. 1, pp. 191-202, 2013.
- [25] M. Bjørling, "Zone append: A new way of writing to zoned storage," USENIX Association, Santa Clara, CA, USA, Tech. Rep., Feb. 2020.
- [26] G. Choi, K. Lee, M. Oh, J. Choi, J. Jhin, and Y. Oh, "A new LSM-style garbage collection scheme for ZNS SSDs," in Proc. 12th USENIX Workshop Hot Topics Storage File Syst. (HotStorage 20), 2020, pp. 1-7.
- [27] RocksDB Wiki, "RocksDB db\_bench options," 2020. [Online]. Available: [https://github.com/EighteenZi/rocksdb\\_wiki/blob/master/Benchmarking-tools.md](https://github.com/EighteenZi/rocksdb_wiki/blob/master/Benchmarking-tools.md)
- [28] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in Proc. 1st ACM Symp. Cloud Comput. (SoCC), Indianapolis, IN, USA, 2010, pp. 143-154.
- [29] B. F. Cooper et al., "Yahoo! Cloud Serving Benchmark (YCSB)," 2010. [Online]. Available: <https://github.com/brianfrankcooper/YCSB>
- [30] Google, "Snappy, a fast compressor/decompressor," 2011. [Online]. Available: <https://github.com/google/snappy/>
- [31] J. Zhu and P. Xie, "D-ZAP: Access Frequency-Aware Dynamic Zone Allocation Optimization for Zoned Storage Devices," in Proc. 6th Int. Conf. Inf. Sci.

Parallel Distrib. Syst. (ISPDS), 2025, pp. 152-156.

- [32] C. Lee, S. Lee, G. Moon, H. Kim, D. An, and D. Kang, "ReZNS: Energy and performance-optimal mapping mechanism for ZNS SSD," *Appl. Sci.*, vol. 14, no. 21, p. 9717, 2024.
- [33] S. Dong, M. Callaghan, L. Galanis, D. Borthakur, T. Savor, and M. Strum, "Optimizing space amplification in RocksDB," in *Proc. Conf. Innov. Data Syst. Res. (CIDR)*, Chaminade, CA, USA, vol. 3, 2017, p. 3.

# Optimizing Zone Management for ZNS-Based Key-Value Stores via SST File Lifetime Awareness

JaeHong Bae

Department of Information Convergence Engineering  
The Graduate School  
Pusan National University  
Abstract

With the proliferation of data-intensive applications and the increasing demand for high-performance storage devices, Zoned Namespace (ZNS) SSDs have emerged as a next-generation storage interface to overcome the limitations of conventional block interfaces. In particular, RocksDB, developed by Meta (formerly Facebook), has become a standard for modern storage engines, providing high write performance based on the Log-Structured Merge-Tree (LSM-Tree) architecture. However, ZenFS—the ZNS-specific storage backend for RocksDB—suffers from zone allocation failures where available zones are prematurely exhausted despite sufficient physical space due to rigid zone management policies. This issue is identified as a critical factor that undermines system availability and device reliability.

To address these limitations, this paper proposes SLA-LZR, a novel user-level zone management policy that considers data lifetime characteristics. The proposed mechanism consists of two core policies. First, the Strong Lifetime Allocation (SLA) policy aggregates SST files with the same lifetime into a single zone, aligning the timing of data invalidation to maximize zone reset efficiency and ensure the timely acquisition of free zones. Second, the Lazy Zone Reset (LZR) policy reflects the short-term lifetime characteristics of RocksDB log files. It employs an adaptive mechanism that postpones resets if a zone is not completely filled even when no valid data remains, thereby preventing unnecessary device wear.

To verify the effectiveness of the proposed policies, we conducted RocksDB db\_bench and YCSB benchmarks in a FEMU-based ZNS SSD environment.

Experimental results demonstrate that SLA-LZR reduced the amount of data migration during garbage collection (GC) by approximately 93.5% compared to the existing ZenFS, leading to a 40.6% improvement in total system throughput. Furthermore, it mitigated the write stall phenomenon by 13.8% by minimizing storage I/O blocking and significantly extended the device's expected lifespan by reducing the number of zone resets by 43.7%. Notably, in the YCSB evaluation, SLA-LZR demonstrated optimal resource management performance by not triggering a single garbage collection event throughout the entire experimental process.

In conclusion, this study presents a practical approach to simultaneously improve the performance and lifespan of ZNS SSD-based next-generation storage systems by effectively reflecting application-level data semantics in storage management policies.